

SIMATS Online Programming Lab
DS PROGRAMMING

Monica Santamil K
1052431198

QUESTIONS 10 / 10 completed
Linear Search
Create a C program to perform Linear Search on an array of 10 integers. The program should allow manual input or random generation of array elements. It must display the first position of the element (1-based indexing), total comparisons made, search status (Found/Not Found).

PROGRAM EDITOR

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <time.h>
4 int main(){
5     int arr[10];
6     int choice,target;
7     int found =0, position =-1, comparisons =0;
8     clock_t start_time, end_time;
9     double time_taken;
10    srand(time(NULL));
11    printf("choose array generation method:\n");
12    printf("1.Manual Input\n");
13    printf("2.Random generation\n");
14    printf("%d",&choice);
15    if(choice==1){
16        printf("Enter 10 integers:\n");
17        for(int i = 0;i<10;i++){
18            scanf("%d",&arr[i]);
19        }
20    }else{
21        printf("Randomly generated array elements: ");

```

INPUT
2
45

OUTPUT
choose array generation method:
1.Manual Input
2.Random generation
-2596Randomly generated array elements: 2063579237412762786
Enter an element to search:

Screenshot 11

SIMATS Online Programming Lab
DS PROGRAMMING

Monica Santamil K
1052431198

QUESTIONS 10 / 10 completed
Binary Search
Write a C program implementing binary search on a sorted array of 15 integers. The program must: (1) accept an unsorted array and sort it first, (2) perform binary search, (3) display search iterations with mid-point calculations, (4) show final result with comparison count, and (5)

PROGRAM EDITOR

```

1 #include<stdio.h>
2 int main(){
3     int a[100],n,i,key;
4     int low = 0,high,mid,found = 0;
5     printf("Enter the number of elements:");
6     scanf("%d",&n);
7     printf("Enter sorted elements:\n");
8     for(i = 0;i<n;i++)
9         scanf("%d",&a[i]);
10    printf("Enter element to search:");
11    scanf("%d",&key);
12    high=n-1;
13    while(low <= high){
14        mid =(low + high)/2;
15        if(a[mid]==key){
16            found =1;
17            printf("Element found at position %d",mid+1);
18            break;
19        }
20        else if(key < a[mid])
21            high = mid -1;

```

INPUT
6
2 4 6 8 10 12
6

OUTPUT
Enter the number of elements:Enter sorted elements:
enter element to search:Element found at position 3

Screenshot 12

SIMATS Online Programming Lab
DS PROGRAMMING

Monica Santamil K
1052431198

QUESTIONS 10 / 10 completed
Binary Search- Iter.
Read n daily temperatures into an array, sort ascending using any method. Use binary search (iterative) to find first day >=30°C and last day <=25°C. Display temperatures, indices (0-based), or "Not found". Validate sorted: n<=50. Sample Input: 6
22 28 31 24 29 32
Sample Output (after

PROGRAM EDITOR

```

1 #include <stdio.h>
2
3 int main() {
4     int n;
5     if (scanf("%d", &n) != 1 || n <= 0 || n > 50) {
6         return 0;
7     }
8
9     int a[50];
10    for (int i = 0; i < n; i++) {
11        scanf("%d", &a[i]);
12    }
13
14    // Sort ascending (Bubble Sort)
15    for (int i = 0; i < n - 1; i++) {
16        for (int j = 0; j < n - i - 1; j++) {
17            if (a[j] > a[j + 1]) {
18                int temp = a[j];
19                a[j] = a[j + 1];
20                a[j + 1] = temp;
21            }

```

INPUT
6
22 28 31 24 29 32

OUTPUT
22 24 28 29 31 32
First >= 30: 31 at index 4
Last <= 25: 24 at index 1

Screenshot 13

SIMATS Online Programming Lab
DS PROGRAMMING

QUESTIONS 10 / 10 completed

Linear Search - Sort

Read n book page counts into an array. Sort ascending using any method. Use linear search to find the first book having >= 300 pages and the last book having <= 150 pages. Display page counts and indices (0-based), or "Not found". Validate sorted order: n<=50. Sample Input: 7

PROGRAM EDITOR

```

1 #include <stdio.h>
2
3 int main() {
4     int n;
5     if (scanf("%d", &n) != 1 || n <= 0 || n > 50) {
6         return 0;
7     }
8
9     int a[50];
10    for (int i = 0; i < n; i++) {
11        scanf("%d", &a[i]);
12    }
13
14    // Sort ascending (Bubble Sort)
15    for (int i = 0; i < n - 1; i++) {
16        for (int j = 0; j < n - i - 1; j++) {
17            if (a[j] > a[j + 1]) {
18                int temp = a[j];
19                a[j] = a[j + 1];
20                a[j + 1] = temp;
21            }
22        }
23    }
24
25    // Find first book with >= 300 pages
26    int first_index = -1;
27    for (int i = 0; i < n; i++) {
28        if (a[i] >= 300) {
29            first_index = i;
30            break;
31        }
32    }
33
34    // Find last book with <= 150 pages
35    int last_index = -1;
36    for (int i = n - 1; i >= 0; i--) {
37        if (a[i] <= 150) {
38            last_index = i;
39            break;
40        }
41    }
42
43    // Output
44    for (int i = 0; i < n; i++) {
45        printf("%d ", a[i]);
46    }
47    printf("\n");
48
49    if (first_index != -1) {
50        printf("First >= 300: %d at index %d\n", a[first_index], first_index);
51    }
52    if (last_index != -1) {
53        printf("Last <= 150: %d at index %d\n", a[last_index], last_index);
54    }
55    return 0;
56 }
```

OUTPUT

```

0 120 140 275 310 443 450
First >= 300: 310 at index 4
Last <= 150: 140 at index 2
```

INPUT

```

7
120 450 275 310 140
```

Screenshot 14

SIMATS Online Programming Lab
DS PROGRAMMING

QUESTIONS 10 / 10 completed

Linear Search - Anu

"Read n strings, use linear search to find all anagrams of a target string (same chars, any order). Display positions (1-based) and matching strings. Case-insensitive; n<=50. Sample Input: 5
1) listen silent
2) race care
3) hello world
4) abc cha
5) Xyz

PROGRAM EDITOR

```

1 #include <stdio.h>
2 #include <string.h>
3
4 int main() {
5     int n;
6     scanf("%d", &n);
7
8     char words[50][100];
9     for (int i = 0; i < n; i++) {
10        scanf("%s", words[i]);
11    }
12
13    char target[100];
14    scanf("%s", target);
15
16    // Count frequencies of target word
17    int target_count[26] = {0};
18    int t_len = strlen(target);
19    for (int i = 0; i < t_len; i++) {
20        char c = target[i];
21        if (c >= 'A' && c <= 'Z') {
22            target_count[c - 'A']++;
23        }
24    }
25
26    // Find anagrams
27    for (int i = 0; i < n; i++) {
28        int word_count[26] = {0};
29        int w_len = strlen(words[i]);
30        for (int j = 0; j < w_len; j++) {
31            char c = words[i][j];
32            if (c >= 'A' && c <= 'Z') {
33                word_count[c - 'A']++;
34            }
35        }
36
37        // Compare frequencies
38        int match = 1;
39        for (int k = 0; k < 26; k++) {
40            if (target_count[k] != word_count[k]) {
41                match = 0;
42            }
43        }
44
45        if (match) {
46            printf("%d: %s\n", i + 1, words[i]);
47        }
48    }
49    return 0;
50 }
```

OUTPUT

```

Security Error: Disallowed code pattern detected.
```

INPUT

```

5
listen silent
race care
hello world
abc cha
Xyz
```

Screenshot 15

SIMATS Online Programming Lab
DS PROGRAMMING

QUESTIONS 10 / 10 completed

Linear Search on A

"Develop a C program to perform Linear Search on an array of N integers. Accept array size dynamically. Sample Input/Output
Enter size: 5
Array: 10 20 30 40 50
Enter target: 40
Comparison Trace:
10 != 40
20 != 40
30 != 40
40 = 40

PROGRAM EDITOR

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main() {
5     int n, target;
6
7     printf("Enter size: ");
8     if (scanf("%d", &n) != 1 || n <= 0) {
9         return 0;
10    }
11
12    // Dynamic memory allocation
13    int *arr = (int *)malloc(n * sizeof(int));
14    if (arr == NULL) {
15        return 0;
16    }
17
18    printf("Array: ");
19    for (int i = 0; i < n; i++) {
20        scanf("%d", &arr[i]);
21    }
22
23    printf("Enter target: ");
24    if (scanf("%d", &target) != 1 || target <= 0) {
25        return 0;
26    }
27
28    // Linear Search
29    for (int i = 0; i < n; i++) {
30        if (arr[i] == target) {
31            printf("Found at index %d\n", i);
32            return 0;
33        }
34    }
35
36    printf("Not found\n");
37    return 0;
38 }
```

OUTPUT

```

Enter size: Array: Enter target: Comparison Trace:
10 != 40
20 != 40
30 != 40
40 = 40
```

INPUT

```

5
10 20 30 40 50
40
```

Screenshot 16

SIMATS Online Programming Lab
DS PROGRAMMING

QUESTIONS 10 / 10 completed

Binary Search - Arr

BINARY SEARCH - ARRAY
Write a C program to perform Binary Search on a sorted array of 10 integers. The program should accept the array elements and a target value from the user. It must display the position of the element and the total number of comparisons made.
Sample Input:
Enter 10 sorted integers:

PROGRAM EDITOR

```

1 #include <stdio.h>
2
3 int main() {
4     int arr[10];
5     int target;
6
7     printf("Enter 10 sorted integers:\n");
8     for (int i = 0; i < 10; i++) {
9         scanf("%d", &arr[i]);
10    }
11
12    printf("Enter target value: ");
13    scanf("%d", &target);
14
15    int low = 0, high = 9;
16    int comparisons = 0;
17    int pos = -1;
18
19    while (low <= high) {
20        comparisons++;
21        int mid = (low + high) / 2;

```

OUTPUT

```

Enter 10 sorted integers:
Enter target value: Element not found
Total comparisons: 3

```

INPUT

```

5 12 18 23 34 45 56
67

```

Screenshot 17

SIMATS Online Programming Lab
DS PROGRAMMING

QUESTIONS 10 / 10 completed

Linear Search - Tem

LINEAR SEARCH - TEMPERATURE
"Read n daily temperatures into an array. Use Linear Search to find the last day whose temperature is less than 20°C. Display temperatures, index (0-based), and value. If not found, print "Not found". Validate n ≤ 50.
Sample Input: 7
18 25 21 19 30 17 24

PROGRAM EDITOR

```

1 #include <stdio.h>
2
3 int main() {
4     int n;
5     if (scanf("%d", &n) != 1 || n <= 0 || n > 50) {
6         return 0;
7     }
8
9     int temp[50];
10    for (int i = 0; i < n; i++) {
11        scanf("%d", &temp[i]);
12    }
13
14    // Display all temperatures
15    for (int i = 0; i < n; i++) {
16        printf("%d ", temp[i]);
17    }
18    printf("\n");
19
20    // Linear search for the last day with temperature < 20
21    int index = -1;

```

OUTPUT

```

18 25 21 19 30 17 24
Index: 5, Value: 17

```

INPUT

```

7
18 25 21 19 30 17 24

```

Screenshot 18

SIMATS Online Programming Lab
DS PROGRAMMING

QUESTIONS 10 / 10 completed

Linear Search - Pric

LINEAR SEARCH - PRICES
"Read n product prices into an array. Sort ascending using any method. Use linear search to find the first price >= \$500 and the last price <= \$200. Display prices and indices (0-based), or "Not found". Validate sorted order; n ≤ 50.
Sample Input: 7
150 650 220 90 500 780 180

PROGRAM EDITOR

```

1 #include <stdio.h>
2
3 int main() {
4     int n;
5     if (scanf("%d", &n) != 1 || n <= 0 || n > 50) {
6         return 0;
7     }
8
9     int a[50];
10    for (int i = 0; i < n; i++) {
11        scanf("%d", &a[i]);
12    }
13
14    // Sort ascending (Bubble Sort)
15    for (int i = 0; i < n - 1; i++) {
16        for (int j = 0; j < n - i - 1; j++) {
17            if (a[j] > a[j + 1]) {
18                int temp = a[j];
19                a[j] = a[j + 1];
20                a[j + 1] = temp;

```

OUTPUT

```

90 150 180 220 500 650 780
First >= 500: 500 at index 4
Last <= 200: 180 at index 2

```

INPUT

```

7
150 650 220 90 500
780 180

```

Screenshot 19

SIMATS Online Programming Lab
DS PROGRAMMING

QUESTIONS 10 / 10 completed
 Bin. Search-Iterativ
BIN. SEARCH-ITERATIVE-TEMP
 "Read n daily temperatures into an array, sort ascending using any method. Use binary search (iterative) to find first day >=30°C and last day <=25°C. Display temperatures, indices (0-based), or "Not found". Validate sorted: n=50. Sample Input 6 22 28 31 24 29 32 Sample Output after

PROGRAM EDITOR C Run Save

```

1 #include <stdio.h>
2
3 int main() {
4     int n;
5     if (scanf("%d", &n) != 1 || n <= 0 || n > 50) {
6         return 0;
7     }
8
9     int a[50];
10    for (int i = 0; i < n; i++) {
11        scanf("%d", &a[i]);
12    }
13
14    // Sort ascending (Bubble Sort)
15    for (int i = 0; i < n - 1; i++) {
16        for (int j = 0; j < n - i - 1; j++) {
17            if (a[j] > a[j + 1]) {
18                int temp = a[j];
19                a[j] = a[j + 1];
20                a[j + 1] = temp;
21            }
22        }
23    }
24
25    // Display sorted array
26    for (int i = 0; i < n; i++) {
27        printf("%d ", a[i]);
28    }
29    printf("\n");
30
31    // Find first day >= 30°C
32    int first = -1;
33    for (int i = 0; i < n; i++) {
34        if (a[i] >= 30) {
35            first = i;
36            break;
37        }
38    }
39
40    // Find last day <= 25°C
41    int last = -1;
42    for (int i = n - 1; i >= 0; i--) {
43        if (a[i] <= 25) {
44            last = i;
45            break;
46        }
47    }
48
49    // Output
50    if (first != -1) {
51        printf("First >=30: index %d (%d)\n", first, a[first]);
52    }
53    if (last != -1) {
54        printf("Last <=25: index %d (%d)\n", last, a[last]);
55    }
56    return 0;
57 }

```

OUTPUT
 22 24 28 29 31 32
 First >=30: index 4 (31)
 Last <=25: index 1 (24)

INPUT
 6
 22 28 31 24 29 32

Screenshot 20

DOUBLY AND CIRCLELY LINKED LIST

SIMATS Online Programming Lab
DS PROGRAMMING

QUESTIONS 10 / 10 completed
 Single Linked List
SINGLE LINKED LIST
 "A college maintains a list of student roll numbers in the order they are registered. New students may join at different stages of the registration process. Develop a C program using a singly linked list to store the roll numbers and perform insertion operations at the beginning, end, and specified position.

PROGRAM EDITOR C Run Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 // Node structure
5 struct Node {
6     int data;
7     struct Node* next;
8 };
9
10 // Function to insert at the beginning
11 struct Node* insertAtBeginning(struct Node* head, int val) {
12     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
13     newNode->data = val;
14     newNode->next = head;
15     return newNode;
16 }
17
18 // Function to insert at the end
19 struct Node* insertAtEnd(struct Node* head, int val) {
20     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
21     newNode->data = val;
22     if (head == NULL) {
23         return newNode;
24     }
25     struct Node* temp = head;
26     while (temp->next != NULL) {
27         temp = temp->next;
28     }
29     temp->next = newNode;
30     return head;
31 }
32
33 // Function to display the list
34 void displayList(struct Node* head) {
35     if (head == NULL) {
36         printf("List is empty\n");
37         return;
38     }
39     struct Node* temp = head;
40     while (temp != NULL) {
41         printf("%d ", temp->data);
42         temp = temp->next;
43     }
44     printf("\n");
45 }
46
47 int main() {
48     struct Node* head = NULL;
49     int val;
50     int choice;
51
52     while (1) {
53         printf("1. Insert at Beginning\n");
54         printf("2. Insert at End\n");
55         printf("3. Display List\n");
56         printf("4. Exit\n");
57         printf("Choice: ");
58         scanf("%d", &choice);
59
60         switch (choice) {
61             case 1:
62                 printf("Enter value: ");
63                 scanf("%d", &val);
64                 head = insertAtBeginning(head, val);
65                 break;
66             case 2:
67                 printf("Enter value: ");
68                 scanf("%d", &val);
69                 head = insertAtEnd(head, val);
70                 break;
71             case 3:
72                 displayList(head);
73                 break;
74             case 4:
75                 return 0;
76             default:
77                 printf("Invalid choice\n");
78         }
79     }
80 }

```

OUTPUT
 101
 103
 102 2

INPUT
 101
 103
 102 2

Screenshot 21

SIMATS Online Programming Lab
DS PROGRAMMING

QUESTIONS 10 / 10 completed
 Deletion Operator
DELETION OPERATIONS
 "A company maintains employee records in a linked list. Employees may leave the organization, requiring their records to be removed. Write a C program to implement deletion operations from the beginning, end, and a specified position. Test Case 1

PROGRAM EDITOR C Run Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node* next;
7 };
8
9 // Function to insert at the end
10 struct Node* insert(struct Node* head, int val) {
11     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
12     newNode->data = val;
13     newNode->next = NULL;
14
15     if (head == NULL) {
16         return newNode;
17     }
18
19     struct Node* temp = head;
20     while (temp->next != NULL) {
21         temp = temp->next;
22     }
23     temp->next = newNode;
24     return head;
25 }
26
27 // Function to delete from beginning
28 struct Node* deleteFromBeginning(struct Node* head) {
29     if (head == NULL) {
30         return NULL;
31     }
32     struct Node* temp = head;
33     if (temp->next != NULL) {
34         temp = temp->next;
35         return temp;
36     }
37     return NULL;
38 }
39
40 // Function to delete from end
41 struct Node* deleteFromEnd(struct Node* head) {
42     if (head == NULL) {
43         return NULL;
44     }
45     struct Node* temp = head;
46     if (temp->next == NULL) {
47         return NULL;
48     }
49     struct Node* prev = NULL;
50     while (temp != NULL) {
51         prev = temp;
52         temp = temp->next;
53     }
54     prev->next = NULL;
55     return head;
56 }
57
58 // Function to delete from specified position
59 struct Node* deleteFromPosition(struct Node* head, int pos) {
60     if (head == NULL) {
61         return NULL;
62     }
63     struct Node* temp = head;
64     if (pos == 1) {
65         return deleteFromBeginning(head);
66     }
67     int count = 1;
68     while (temp != NULL) {
69         if (count == pos) {
70             if (temp->next != NULL) {
71                 struct Node* next = temp->next;
72                 prev->next = next;
73                 free(temp);
74                 return head;
75             }
76             return NULL;
77         }
78         prev = temp;
79         temp = temp->next;
80         count++;
81     }
82     return head;
83 }
84
85 int main() {
86     struct Node* head = NULL;
87     int val;
88     int choice;
89     int pos;
90
91     while (1) {
92         printf("1. Insert at End\n");
93         printf("2. Delete from Beginning\n");
94         printf("3. Delete from End\n");
95         printf("4. Delete from Position\n");
96         printf("5. Display List\n");
97         printf("6. Exit\n");
98         printf("Choice: ");
99         scanf("%d", &choice);
100
101         switch (choice) {
102             case 1:
103                 printf("Enter value: ");
104                 scanf("%d", &val);
105                 head = insert(head, val);
106                 break;
107             case 2:
108                 head = deleteFromBeginning(head);
109                 break;
110             case 3:
111                 head = deleteFromEnd(head);
112                 break;
113             case 4:
114                 printf("Enter position: ");
115                 scanf("%d", &pos);
116                 head = deleteFromPosition(head, pos);
117                 break;
118             case 5:
119                 displayList(head);
120                 break;
121             case 6:
122                 return 0;
123             default:
124                 printf("Invalid choice\n");
125         }
126     }
127 }

```

OUTPUT
 20

INPUT
 1 10
 1 20
 1 30
 1 40
 2
 3
 4 2
 5
 6

Screenshot 22

SIMATS Online Programming Lab
DS PROGRAMMING

QUESTIONS 10 / 10 completed

Linked List

LINKED LIST
 *A library stores book identification numbers in a linked list.
 Develop a C program that searches for a given element and displays its position if found.
 Test Case 1
 Input
 List: 10 → 20 → 30 → 40
 Search Element: 30

PROGRAM EDITOR

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node* next;
7 };
8
9 // Function to append a node at the end of the list
10 struct Node* append(struct Node* head, int val) {
11     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
12     newNode->data = val;
13     newNode->next = NULL;
14
15     if (head == NULL) {
16         return newNode;
17     }
18
19     struct Node* temp = head;
20     while (temp->next != NULL) {
21         temp = temp->next;

```

OUTPUT

Element found at position 3

INPUT

4
 10 20 30 40
 30

Screenshot 23

SIMATS Online Programming Lab
DS PROGRAMMING

QUESTIONS 10 / 10 completed

Singly Linked List-4

SINGLY LINKED LIST-REVERSE
 *In a logistics application, delivery locations are stored in a linked list. Sometimes the route needs to be traversed in reverse order. Write a C program to reverse a singly linked list and display the reversed list.
 Test Case 1
 Input

PROGRAM EDITOR

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node* next;
7 };
8
9 // Function to insert a node at the end
10 struct Node* append(struct Node* head, int val) {
11     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
12     newNode->data = val;
13     newNode->next = NULL;
14
15     if (head == NULL) {
16         return newNode;
17     }
18
19     struct Node* temp = head;
20     while (temp->next != NULL) {
21         temp = temp->next;

```

OUTPUT

40 30 20 10

INPUT

10 20 30 40

Screenshot 24

SIMATS Online Programming Lab
DS PROGRAMMING

QUESTIONS 10 / 10 completed

Singly Linked List-4

SINGLY LINKED LIST-NODES
 *A hospital maintains patient records using a linked list. To generate reports, the system needs to determine the total number of records currently stored. Write a C program to count and display the total number of nodes in a singly linked list.
 Test Case 1
 Input

PROGRAM EDITOR

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node* next;
7 };
8
9 // Function to insert a node at the end
10 struct Node* append(struct Node* head, int val) {
11     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
12     newNode->data = val;
13     newNode->next = NULL;
14
15     if (head == NULL) {
16         return newNode;
17     }
18
19     struct Node* temp = head;
20     while (temp->next != NULL) {
21         temp = temp->next;

```

OUTPUT

4

INPUT

10 20 30 40

Screenshot 25

SIMATS Online Programming Lab
DS PROGRAMMING

QUESTIONS 10/10 completed
Single Sorted Linked List

Two departments maintain separate sorted lists of student IDs. For centralized processing, both lists need to be merged while preserving the sorted order. Write a C program to merge two sorted singly linked lists into a single sorted linked list.

Test Case 1

Input

PROGRAM EDITOR

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node* next;
7 };
8
9 // Function to append a node at the end
10 struct Node* append(struct Node* head, int val) {
11     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
12     newNode->data = val;
13     newNode->next = NULL;
14
15     if (head == NULL) {
16         return newNode;
17     }
18
19     struct Node* temp = head;
20     while (temp->next != NULL) {
21         temp = temp->next;
22     }
23     temp->next = newNode;
24     return head;
25 }

```

OUTPUT

INPUT
Your INPUT goes here!

Screenshot 26

SIMATS Online Programming Lab
DS PROGRAMMING

QUESTIONS 10/10 completed
Sorted Linked List-2

A database contains sorted customer IDs, but due to data-entry errors, duplicate records have been created. Develop a C program that removes duplicate nodes from a sorted linked list and displays the updated list.

Test Case 1

Input

PROGRAM EDITOR

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 // Definition of the singly linked list node
5 struct Node {
6     int data;
7     struct Node* next;
8 };
9
10 // Function to insert a new node at the end of the list
11 void appendNode(struct Node** head_ref, int new_data) {
12     struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
13     struct Node* last = *head_ref;
14
15     new_node->data = new_data;
16     new_node->next = NULL;
17
18     if (*head_ref == NULL) {
19         *head_ref = new_node;
20     }
21     return;
22 }

```

OUTPUT
10 -> 20 -> 30 -> NULL

INPUT
10 -> 10 -> 20 -> 20 -> 30 -> 30

Screenshot 27

SIMATS Online Programming Lab
DS PROGRAMMING

QUESTIONS 10/10 completed
Circular Linked List

In a round-robin CPU scheduling system, processes are executed cyclically. Such applications can be represented using a circular linked list. Write a C program to create a circular linked list and display all nodes by traversing the list exactly once. Write a C program to create a circular linked list and traverse all

PROGRAM EDITOR

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node* next;
7 };
8
9
10 struct Node* insertEnd(struct Node* head, int data) {
11     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
12     newNode->data = data;
13
14     if (head == NULL) {
15         newNode->next = newNode;
16         return newNode;
17     }
18
19     struct Node* temp = head;
20     while (temp->next != head) {
21         temp = temp->next;
22     }
23     temp->next = newNode;
24     return head;
25 }

```

OUTPUT
10 -> Back to Head

INPUT
10 -> 20 -> 30 -> 40

Screenshot 28

SIMATS Online Programming Lab
DS PROGRAMMING

Monica Santamil K
9252431588

QUESTIONS 10 / 10 completed
Doubly Linked List
DOUBLY LINKED LIST
"A music player maintains a playlist where users can move both forward and backward between songs. This can be efficiently implemented using a doubly linked list. Write a C program to create a doubly linked list and display nodes in forward and backward directions. Test Case 1"

PROGRAM EDITOR

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node* prev;
7     struct Node* next;
8 };
9
10
11 struct Node* insertInfd(struct Node* head, int data) {
12     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
13     newNode->data = data;
14     newNode->next = NULL;
15     newNode->prev = NULL;
16
17     if (head == NULL) {
18         return newNode;
19     }
20
21     struct Node* temp = head;

```

OUTPUT

```

Forward Traversal : 10
Backward Traversal: 10

```

INPUT

```

10 -> 20 -> 30 -> 40

```

Screenshot 29

SIMATS Online Programming Lab
DS PROGRAMMING

Monica Santamil K
9252431588

QUESTIONS 10 / 10 completed
Polynomials
POLYNOMIALS
"In scientific computing applications, mathematical polynomials are often represented using linked lists where each node stores a coefficient and exponent. Write a C program to represent two polynomials using linked lists and perform polynomial addition."

PROGRAM EDITOR

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4
5 struct Node {
6     int coeff;
7     int exp;
8     struct Node* next;
9 };
10
11
12 struct Node* insertTerm(struct Node* head, int coeff, int exp) {
13     if (coeff == 0) return head;
14
15     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
16     newNode->coeff = coeff;
17     newNode->exp = exp;
18     newNode->next = NULL;
19
20     if (head == NULL || exp > head->exp) {
21         newNode->next = head;

```

OUTPUT

```

Resultant Polynomial:
9x4 + 9x + 3

```

INPUT

```

Polynomial 1: 5x4 + 3x + 2
Polynomial 2: 4x4 + 6x + 1

```

Screenshot 30

STACK DATASTRUCTURE:

SIMATS Online Programming Lab
DS PROGRAMMING

Monica Santamil K
9252431588

QUESTIONS 10 / 10 completed
Stack using Array
STACK USING ARRAY
"A web browser maintains a history of recently visited pages. Write a C program to implement a stack using an array and perform Push, Pop, Peek, and Display operations while handling overflow and underflow conditions. Test Case 1"

PROGRAM EDITOR

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define MAX 100
5
6 int stack[MAX];
7 int top = -1;
8 int capacity = 0;
9
10 void push(int value) {
11     if (top == capacity - 1) {
12         printf("Stack Overflow\n");
13         return;
14     }
15     stack[++top] = value;
16 }
17
18 int pop() {
19     if (top == -1) {
20         printf("Stack Underflow\n");
21         return -1;

```

OUTPUT

```

Stack Elements:
30
20
10

```

INPUT

```

stack size = 5
push: 10,20,30
Display

```

Screenshot 31

SIMATS Online Programming Lab
DS PROGRAMMING

Monica Santamil K
192524315RB

QUESTIONS 10 / 10 completed

Stack using Linked

STACK USING LINKED LIST

"A document editor maintains recently performed actions to support the Undo feature. Implement a stack using a linked list and perform push, pop, and display operations.
Test Case 1

Input
Push: 100, 200, 300
Display

PROGRAM EDITOR

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node* next;
7 };
8
9 struct Node* top = NULL;
10
11 void push(int value) {
12     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
13     if (newNode == NULL) {
14         return;
15     }
16     newNode->data = value;
17     newNode->next = top;
18     top = newNode;
19 }
20
21 int pop() {

```

OUTPUT

300 -> 200 -> 100 -> NULL

INPUT

push: 100,200,300
Display

Screenshot 32

SIMATS Online Programming Lab
DS PROGRAMMING

Monica Santamil K
192524315RB

QUESTIONS 10 / 10 completed

Stack -Postfix Expr

STACK -POSTFIX EXPRESSION

"Write a C program that uses a stack to evaluate a postfix expression and display the result.
Postfix expressions are commonly used in compilers because they eliminate the need for parentheses.
Input
23*5+

Output

PROGRAM EDITOR

```

1 #include <stdio.h>
2 #include <ctype.h>
3 #include <stdlib.h>
4
5 #define MAX 100
6
7 int stack[MAX];
8 int top = -1;
9
10 void push(int val) {
11     if (top < MAX - 1) {
12         stack[++top] = val;
13     }
14 }
15
16 int pop() {
17     if (top >= 0) {
18         return stack[top--];
19     }
20     return 0;
21 }

```

OUTPUT

Result = 11

INPUT

23*5+

Screenshot 33

SIMATS Online Programming Lab
DS PROGRAMMING

Monica Santamil K
192524315RB

QUESTIONS 10 / 10 completed

Stack -Infix to Post

STACK -INFIX TO POSTFIX

"Arithmetic expressions entered by programmers are generally in infix form. Before evaluation, many systems convert them into postfix notation. Write a C program using a stack to convert an infix expression into its equivalent postfix expression.
Test Case 1

PROGRAM EDITOR

```

1 #include <stdio.h>
2 #include <ctype.h>
3 #include <string.h>
4
5 #define MAX 100
6
7 char stack[MAX];
8 int top = -1;
9
10 void push(char c) {
11     if (top < MAX - 1) {
12         stack[++top] = c;
13     }
14 }
15
16 char pop() {
17     if (top >= 0) {
18         return stack[top--];
19     }
20     return -1;
21 }

```

OUTPUT

ABC*+

INPUT

A*B^C

Screenshot 34

SIMATS Online Programming Lab
DS PROGRAMMING

Monica Sentamil K
19252431588

QUESTIONS 10 / 10 completed
Stack- Balanced Pa

STACK- BALANCED PARENTHESES

Compilers must verify whether all opening parentheses have corresponding closing parentheses. Write a C program using a stack to check whether an expression contains balanced parentheses.

Input: (A+B)*(C-D)

Output:

PROGRAM EDITOR

```

1 #include <stdio.h>
2 #include <string.h>
3
4 #define MAX 100
5
6 char stack[MAX];
7 int top = -1;
8
9 void push(char c) {
10     if (top < MAX - 1) {
11         stack[++top] = c;
12     }
13 }
14
15 char pop() {
16     if (top >= 0) {
17         return stack[top--];
18     }
19     return '\0';
20 }

```

OUTPUT
Balanced Expression

INPUT
(A+B)*(C-D)

Screenshot 35

SIMATS Online Programming Lab
DS PROGRAMMING

Monica Sentamil K
19252431588

QUESTIONS 10 / 10 completed
Stack - Reverse Stri

STACK - REVERSE STRING

A text-processing application requires reversing user-entered strings. Write a C program that uses a stack to reverse a given string.

Input: ERUTCURTSATAD

DATASTRUCTURE

Output:

PROGRAM EDITOR

```

1 #include <stdio.h>
2 #include <string.h>
3
4 #define MAX 100
5
6 struct Stack {
7     int top;
8     char items[MAX];
9 };
10
11 void push(struct Stack* s, char ch) {
12     if (s->top < MAX - 1) {
13         s->items[++(s->top)] = ch;
14     }
15 }
16
17 char pop(struct Stack* s) {
18     if (s->top >= 0) {
19         return s->items[(s->top)--];
20     }
21 }

```

OUTPUT
ERUTCURTSATAD

INPUT
DATASTRUCTURE

Screenshot 36

SIMATS Online Programming Lab
DS PROGRAMMING

Monica Sentamil K
19252431588

QUESTIONS 10 / 10 completed
Stack- Decimal to i

STACK- DECIMAL TO BINARY

Computer systems internally represent numbers in binary format. Write a C program that uses a stack to convert a decimal number into its binary equivalent.

Input: 13

Output: 1101

PROGRAM EDITOR

```

1 #include <stdio.h>
2
3 #define MAX 100
4
5 struct Stack {
6     int top;
7     int items[MAX];
8 };
9
10 void push(struct Stack* s, int val) {
11     if (s->top < MAX - 1) {
12         s->items[++(s->top)] = val;
13     }
14 }
15
16 int pop(struct Stack* s) {
17     if (s->top >= 0) {
18         return s->items[(s->top)--];
19     }
20     return -1;
21 }

```

OUTPUT
1101

INPUT
13

Screenshot 37

SIMATS Online Programming Lab
DS PROGRAMMING

Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed
Tower of Hanoi Prc
TOWER OF HANOI PROBLEM
The Tower of Hanoi is a classic problem demonstrating recursion and stack behavior. Write a C program to solve the Tower of Hanoi problem and display the sequence of moves required to transfer all disks.
Input
Number of Disks = 3

PROGRAM EDITOR

```

1 #include <stdio.h>
2
3 void towerOfHanoi(int n, char from_rod, char to_rod, char aux_rod) {
4     if (n == 0) {
5         return;
6     }
7     towerOfHanoi(n - 1, from_rod, aux_rod, to_rod);
8     printf("Move Disk %d from %c to %c\n", n, from_rod, to_rod);
9     towerOfHanoi(n - 1, aux_rod, to_rod, from_rod);
10 }
11
12 int main() {
13     int n;
14     if (scanf("%d", &n) != 1) {
15         if (scanf("Number of Disks = %d", &n) != 1) {
16             scanf("%d", &n);
17         }
18     }
19     towerOfHanoi(n, 'A', 'C', 'B');
20 }

```

OUTPUT

```

Move Disk 1 from A to C
Move Disk 2 from A to B
Move Disk 1 from C to B
Move Disk 3 from A to C
Move Disk 1 from B to A

```

INPUT
Number of Disks = 3

Screenshot 38

SIMATS Online Programming Lab
DS PROGRAMMING

Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed
Stack - Browser His
STACK - BROWSER HISTORY
A web browser stores visited web pages in a stack to enable navigation to previously visited pages. Write a C program that simulates browser history using a stack and supports back navigation.
Input
Visit:
Google

PROGRAM EDITOR

```

1 #include <stdio.h>
2 #include <string.h>
3
4 #define MAX 100
5 #define LEN 100
6
7 struct Stack {
8     int top;
9     char items[MAX][LEN];
10 };
11
12 void push(struct Stack *s, char *str) {
13     if (s->top < MAX - 1) {
14         strcpy(s->items[++(s->top)], str);
15     }
16 }
17
18 void pop(struct Stack *s) {
19     if (s->top >= 0) {
20         s->top--;
21     }
22 }

```

OUTPUT

```

Current Page:
Youtube

```

INPUT
visit:
Google
Youtube
Wikipedia
Back

Screenshot 39

SIMATS Online Programming Lab
DS PROGRAMMING

Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed
Stack- Undo Opera
STACK- UNDO OPERATION
A text editor provides an Undo feature that allows users to revert the most recent action. Write a C program using a stack to simulate an Undo operation for text editing commands.
Input
Insert A
Insert B
Insert C
Undo

PROGRAM EDITOR

```

1 #include <stdio.h>
2 #include <string.h>
3
4 #define MAX 100
5
6 struct Stack {
7     int top;
8     char items[MAX];
9 };
10
11 void push(struct Stack *s, char ch) {
12     if (s->top < MAX - 1) {
13         s->items[++(s->top)] = ch;
14     }
15 }
16
17 void pop(struct Stack *s) {
18     if (s->top >= 0) {
19         s->top--;
20     }
21 }

```

OUTPUT

```

Current Text: AB

```

INPUT
Insert A
Insert B
Insert C
Undo

Screenshot 40

QUEUE DATASTRUCTURES :

The screenshot shows the SIMATS Online Programming Lab interface. The top header includes the SIMATS Engineering logo, the text "SIMATS Online Programming Lab", and the user's name "Monica Sentamil K" with a dropdown arrow. Below the header, the interface is divided into three main sections: QUESTIONS, PROGRAM EDITOR, and INPUT.

QUESTIONS (10 / 10 completed): A dropdown menu shows "Queue Using Array". The question text reads: "Write a C program to implement a Queue using Arrays with operations such as insertion (enqueue), deletion (dequeue), display, and peek. The program should simulate a ticket booking counter where customers arrive in sequence and are served on a first-come-first-served basis. The queue".

PROGRAM EDITOR: The code editor shows a C program for a queue using arrays. The code is as follows:

```
#include <stdio.h>
#include <string.h>
#define MAX 100
int queue[MAX];
int front = -1;
int rear = -1;

void enqueue(int val) {
    if (rear == MAX - 1) {
        return;
    }
    if (front == -1) {
        front = 0;
    }
    queue[++rear] = val;
}

void dequeue() {
    if (front == -1 || front > rear) {
        return;
    }
    front++;
}
```

OUTPUT: The output shows "Queue Elements: 101 102 103" and "Deleted Element: 101".

INPUT: The input field contains the text: "Enqueue 101, Enqueue 102, Enqueue 103, Display, Dequeue, Display, Sample".

Screenshot 41

The screenshot shows the SIMATS Online Programming Lab interface. The top header includes the SIMATS Engineering logo, the text "SIMATS Online Programming Lab", and the user's name "Monica Sentamil K" with a dropdown arrow. Below the header, the interface is divided into three main sections: QUESTIONS, PROGRAM EDITOR, and INPUT.

QUESTIONS (10 / 10 completed): A dropdown menu shows "Queue Using Linke". The question text reads: "Write a C program to implement a Queue using Linked Lists for managing patients waiting in a hospital consultation room. The program should support enqueue, dequeue, display, and front operations without any fixed size limitation. Sample Input: Insert Patient IDs 201, 202, 203, Display, Delete one".

PROGRAM EDITOR: The code editor shows a C program for a queue using linked lists. The code is as follows:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
struct Node {
    int data;
    struct Node *next;
};
struct Node *front = NULL;
struct Node *rear = NULL;

void enqueue(int val) {
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = val;
    newNode->next = NULL;
    if (rear == NULL) {
        front = rear = newNode;
        return;
    }
    rear->next = newNode;
    rear = newNode;
}
```

OUTPUT: The output shows "Queue Elements: 201 202 203", "Deleted Patient ID: 201", and "Remaining Queue: 202 203".

INPUT: The input field contains the text: "Insert Patient IDs 201, 202, 203, Display, Delete one patient, Sample".

Screenshot 42



SIMATS Online Programming Lab
DS PROGRAMMING



QUESTIONS 10 / 10 completed

Double Ended Queue

DOUBLE ENDED QUEUE

Write a C program to implement a Double Ended Queue (Deque) using Arrays where insertion and deletion can be performed from both front and rear ends. Consider a warehouse inventory management system where goods can be added or removed from either side based on priority. Sample Input: Insert Front 100,

PROGRAM EDITOR
C Run Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <ctype.h>
5
6 #define MAX 100
7
8 int deque[MAX];
9 int front = -1, rear = -1;
10
11 int isFull() {
12     return ((front == 0 && rear == MAX - 1) || (front == rear + 1));
13 }
14
15
16 int isEmpty() {
17     return (front == -1);
18 }
19
20
21 void insertFront(int key) {

```

INPUT

Your INPUT goes here!

OUTPUT

Deleted Element: 200
Queue Elements: 50 100

Screenshot 43



SIMATS Online Programming Lab
DS PROGRAMMING



QUESTIONS 10 / 10 completed

Priority Queue Usir

PRIORITY QUEUE USING ARRAYS

Write a C program to implement a Priority Queue using Arrays for processing emergency service requests. Requests with higher priority must be processed before lower-priority requests irrespective of arrival order. Sample Input: Insert (101, Priority 3), Insert (102, Priority 1), Insert (103, Priority 2),

PROGRAM EDITOR
C Run Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define MAX 100
5
6 struct Request {
7     int id;
8     int priority;
9 };
10
11 struct Request pq[MAX];
12 int size = 0;
13
14 void insert(int id, int priority) {
15     if (size >= MAX) {
16         printf("Queue Overflow\n");
17         return;
18     }
19     pq[size].id = id;
20     pq[size].priority = priority;
21     size++;

```

INPUT

Your INPUT goes here!

OUTPUT

Processed Request: 102
Remaining Queue: 101 103

Screenshot 44



SIMATS Online Programming Lab
DS PROGRAMMING


Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed

Queue Data Structi

QUEUE DATA STRUCTURE

Write a C program to simulate a Railway Reservation Waiting List using Queue data structure. Passengers enter the waiting queue and are allocated seats as cancellations occur. The program should maintain passenger details and process requests in FIFO order.
Sample Input: Add Passenger 501, Add

PROGRAM EDITOR

C

Run Save

```

1 #include <stdio.h>
2
3 #define MAX 100
4
5 int queue[MAX];
6 int front = -1, rear = -1;
7
8 void addPassenger(int id) {
9     if (rear == MAX - 1) {
10         printf("Queue Overflow\n");
11         return;
12     }
13     if (front == -1) {
14         front = 0;
15     }
16     rear++;
17     queue[rear] = id;
18 }
19
20 void processCancellation() {
21     if (front == -1 || front > rear) {

```

OUTPUT

```

Passenger 501 Confirmed.
Waiting List: 502

```

INPUT

Your INPUT goes here!

Screenshot 45



SIMATS Online Programming Lab
DS PROGRAMMING


Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed

Queue

QUEUE

Write a C program to implement a Queue for managing online food delivery orders where orders are received and processed in the order they arrive. The program should allow adding new orders, processing completed orders, and displaying pending orders.
Sample Input: Add Orders 1001, 1002,

PROGRAM EDITOR

C

Run Save

```

1 #include <stdio.h>
2
3 #define MAX 100
4
5 int queue[MAX];
6 int front = -1, rear = -1;
7
8 void enqueue(int order) {
9     if (rear == MAX - 1) {
10         printf("Queue Overflow\n");
11         return;
12     }
13     if (front == -1) {
14         front = 0;
15     }
16     rear++;
17     queue[rear] = order;
18 }
19
20 void dequeue() {
21     if (front == -1 || front > rear) {

```

OUTPUT

```

Processed Order: 1001
Pending Orders: 1002 1003

```

INPUT

Your INPUT goes here!

Screenshot 46

SIMATS
ENGINEERING

SIMATS Online Programming Lab
DS PROGRAMMING

Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed
 Queue Using Linke
QUEUE USING LINKED LISTS -2
 Write a C program to implement a Queue using Linked Lists for managing network packets in a router. Packets should be transmitted in the order received, and the program must support dynamic packet insertion and removal. Sample Input: Insert Packets 11, 12, 13, Transmit Packet. Sample Output:

PROGRAM EDITOR C Run Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node* next;
7 };
8
9 struct Node *front = NULL, *rear = NULL;
10
11 void enqueue(int val) {
12     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
13     newNode->data = val;
14     newNode->next = NULL;
15
16     if (rear == NULL) {
17         front = rear = newNode;
18         return;
19     }
20
21     rear->next = newNode;

```

OUTPUT
 Transmitted Packet: 11
 Remaining Packets: 12 13

INPUT
 Your INPUT goes here!

Screenshot 47

TREES

TREE TRAVERSAL:

SIMATS
ENGINEERING

SIMATS Online Programming Lab
DS PROGRAMMING

Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed
 Binary Tree-Inorder
BINARY TREE-INORDER REVERSAL
 Write a C program to create a Binary Tree and perform Recursive Inorder Traversal. Consider an organizational hierarchy management system where employee records are stored in a binary tree structure. The program should create the tree dynamically and display employee IDs

PROGRAM EDITOR C Run Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node* left;
7     struct Node* right;
8 };
9
10 struct Node* createNode(int data) {
11     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
12     newNode->data = data;
13     newNode->left = newNode->right = NULL;
14     return newNode;
15 }
16
17 struct Node* insert(struct Node* root, int data) {
18     if (root == NULL)
19         return createNode(data);
20     if (data < root->data)
21         root->left = insert(root->left, data);

```

OUTPUT
 Inorder Traversal: 20 30 40 50 60 70 80

INPUT
 Your INPUT goes here!

Screenshot 48

SIMATS
ENGINEERING

SIMATS Online Programming Lab
DS PROGRAMMING

Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed
 BT- Rec. Preorder T
BT- REC. PREORDER TRAVERSE
 Write a C program to create a Binary Tree and perform Recursive Preorder Traversal. Consider a file backup application where directories and subdirectories are represented as a tree structure. The program should display the order in which folders are visited during the backup process using preorder traversal.

PROGRAM EDITOR

C
 Run Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node* left;
7     struct Node* right;
8 };
9
10 struct Node* createNode(int data) {
11     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
12     newNode->data = data;
13     newNode->left = newNode->right = NULL;
14     return newNode;
15 }
16
17 struct Node* insert(struct Node* root, int data) {
18     if (root == NULL)
19         return createNode(data);
20     if (data < root->data)
21         root->left = insert(root->left, data);
    
```

OUTPUT
 Preorder Traversal: 10 5 3 7 15 12 18

INPUT
 Your INPUT goes here!

Screenshot 49

SIMATS
ENGINEERING

SIMATS Online Programming Lab
DS PROGRAMMING

Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed
 BT- Rec. Postorder
BT- REC. POSTORDER TRAVERSE
 Write a C program to create a Binary Tree and perform Recursive Postorder Traversal. Consider a software uninstallation utility where dependent components must be removed before the parent application. The program should display the deletion order using postorder traversal. Sample Input: Enter Nodes: 25

PROGRAM EDITOR

C
 Run Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node* left;
7     struct Node* right;
8 };
9
10 struct Node* createNode(int data) {
11     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
12     newNode->data = data;
13     newNode->left = newNode->right = NULL;
14     return newNode;
15 }
16
17 struct Node* insert(struct Node* root, int data) {
18     if (root == NULL)
19         return createNode(data);
20     if (data < root->data)
21         root->left = insert(root->left, data);
    
```

OUTPUT
 Preorder Traversal: 10 5 3 7 15 12 18

INPUT
 Your INPUT goes here!

Screenshot 50



SIMATS Online Programming Lab
DS PROGRAMMING

 Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed

BST- In,Pre,Post Trv

BST- IN,PRE,POST TRAVERSE
Write a C program to create a Binary Search Tree (BST) and perform Inorder, Preorder, and Postorder Traversals. Consider a student record management system where roll numbers are stored in a BST and different traversals are used for various reporting purposes. Sample Input: Enter Nodes: 45 25 65 15 35 55 75.

PROGRAM EDITOR

C

Run Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node* left;
7     struct Node* right;
8 };
9
10 struct Node* createNode(int data) {
11     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
12     newNode->data = data;
13     newNode->left = newNode->right = NULL;
14     return newNode;
15 }
16
17 struct Node* insert(struct Node* root, int data) {
18     if (root == NULL)
19         return createNode(data);
20     if (data < root->data)
21         root->left = insert(root->left, data);

```

OUTPUT

```

Inorder: 15 25 35 45 55 65 75
Preorder: 45 25 15 35 65 55 75
Postorder: 15 35 25 55 75 65 45

```

INPUT

Your INPUT goes here!

Screenshot 51



SIMATS Online Programming Lab
DS PROGRAMMING

 Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed

BST- Search Eleme

BST- SEARCH ELEMENT
Write a C program to insert elements into a Binary Search Tree and search for a specific element. Consider an online product catalog where product IDs are stored in a BST for efficient retrieval. The program should indicate whether a given product ID exists in the tree. Sample Input: Nodes: 50 30 70 20 40 60 80. Search

PROGRAM EDITOR

C

Run Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node* left;
7     struct Node* right;
8 };
9
10 struct Node* createNode(int data) {
11     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
12     newNode->data = data;
13     newNode->left = newNode->right = NULL;
14     return newNode;
15 }
16
17 struct Node* insert(struct Node* root, int data) {
18     if (root == NULL)
19         return createNode(data);
20     if (data < root->data)
21         root->left = insert(root->left, data);

```

OUTPUT

```

Element 60 Found in the Binary Search Tree.

```

INPUT

Your INPUT goes here!

Screenshot 52

SIMATS
ENGINEERING

SIMATS Online Programming Lab
DS PROGRAMMING

Monica Sentamil K
192524315R8

QUESTIONS 10 / 10 completed
BST- Finding Height
BST- FINDING HEIGHT

Write a C program to find the height of a Binary Tree. Consider a network routing structure where devices are arranged hierarchically and the tree height represents the maximum communication depth. The program should determine the height of the constructed tree. Sample Input: Nodes: 1 2 3 4 5 6 7.

PROGRAM EDITOR C Run Save


```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node* left;
7     struct Node* right;
8 };
9
10 struct Node* createNode(int data) {
11     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
12     newNode->data = data;
13     newNode->left = newNode->right = NULL;
14     return newNode;
15 }
16
17 struct Node* insert(struct Node* root, int data) {
18     if (root == NULL)
19         return createNode(data);
20     if (data < root->data)
21         root->left = insert(root->left, data);
        
```

OUTPUT

```

Height of Binary Tree: 6
        
```

INPUT
 Your INPUT goes here!

Screenshot 53

SIMATS
ENGINEERING

SIMATS Online Programming Lab
DS PROGRAMMING

Monica Sentamil K
192524315R8

QUESTIONS 10 / 10 completed
BST- Count Nodes
BST- COUNT NODES

Write a C program to count the total number of nodes, leaf nodes, and non-leaf nodes in a Binary Tree. Consider a company's organizational structure where leaf nodes represent employees without subordinates and non-leaf nodes represent managers. Sample Input: Nodes: 50 30 70 20 40 60 80. Sample

PROGRAM EDITOR C Run Save


```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node* left;
7     struct Node* right;
8 };
9
10 struct Node* createNode(int data) {
11     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
12     newNode->data = data;
13     newNode->left = newNode->right = NULL;
14     return newNode;
15 }
16
17 struct Node* insert(struct Node* root, int data) {
18     if (root == NULL)
19         return createNode(data);
20     if (data < root->data)
21         root->left = insert(root->left, data);
        
```

OUTPUT

```

Total Nodes: 7.
Leaf Nodes: 4.
Non-Leaf Nodes: 3.
        
```

INPUT
 Your INPUT goes here!

Screenshot 54



SIMATS Online Programming Lab
DS PROGRAMMING



QUESTIONS 10 / 10 completed

BST- Min, Max Val.

BST- MIN, MAX VALUES

Write a C program to find the minimum and maximum values in a Binary Search Tree. Consider a sales database where transaction amounts are stored in a BST and management wants to identify the smallest and largest transactions. Sample Input: Nodes: 100 50 150 25 75 125 175. Sample Output:

C
Run Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node* left;
7     struct Node* right;
8 };
9
10 struct Node* createNode(int data) {
11     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
12     newNode->data = data;
13     newNode->left = newNode->right = NULL;
14     return newNode;
15 }
16
17 struct Node* insert(struct Node* root, int data) {
18     if (root == NULL)
19         return createNode(data);
20     if (data < root->data)
21         root->left = insert(root->left, data);

```

Your INPUT goes here!

Minimum Value: 25
Maximum Value: 175

Screenshot 55



SIMATS Online Programming Lab
DS PROGRAMMING



QUESTIONS 10 / 10 completed

BST- Deletion

BST- DELETION

Write a C program to delete a node from a Binary Search Tree and display the tree using inorder traversal after deletion. Consider a library management system where discontinued book IDs must be removed from the catalog. Sample Input: Nodes: 50 30 70 20 40 60 80. Delete Node: 30. Sample Output:

C
Run Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node* left;
7     struct Node* right;
8 };
9
10 struct Node* createNode(int data) {
11     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
12     newNode->data = data;
13     newNode->left = newNode->right = NULL;
14     return newNode;
15 }
16
17 struct Node* insert(struct Node* root, int data) {
18     if (root == NULL)
19         return createNode(data);
20     if (data < root->data)
21         root->left = insert(root->left, data);

```

Your INPUT goes here!

Inorder Traversal after Deletion: 20 40 50 60 70 80

Screenshot 56



SIMATS Online Programming Lab
DS PROGRAMMING

Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed

BST- Mirroring

BST- MIRRORING

Write a C program to mirror a Binary Tree and display the inorder traversal before and after mirroring. Consider a graphical application where tree structures need to be reflected horizontally for visualization purposes.

Sample Input: Nodes:
10 5 15 3 7 12 18.

Sample Output:
Original Inorder: 3 5 7

PROGRAM EDITOR

C
Run
Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node* left;
7     struct Node* right;
8 };
9
10 struct Node* createNode(int data) {
11     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
12     newNode->data = data;
13     newNode->left = newNode->right = NULL;
14     return newNode;
15 }
16
17 struct Node* insert(struct Node* root, int data) {
18     if (root == NULL)
19         return createNode(data);
20     if (data < root->data)
21         root->left = insert(root->left, data);

```

INPUT

Your INPUT goes here!

OUTPUT

Original Inorder: 3 5 7 10 12 15 18
Mirrored Inorder: 18 15 12 10 7 5 3

Screenshot 57

AVL TREE:



SIMATS Online Programming Lab
DS PROGRAMMING

Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed

BST- Mirroring

BST- MIRRORING

Write a C program to mirror a Binary Tree and display the inorder traversal before and after mirroring. Consider a graphical application where tree structures need to be reflected horizontally for visualization purposes.

Sample Input: Nodes:
10 5 15 3 7 12 18.

Sample Output:
Original Inorder: 3 5 7

PROGRAM EDITOR

C
Run
Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node* left;
7     struct Node* right;
8 };
9
10 struct Node* createNode(int data) {
11     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
12     newNode->data = data;
13     newNode->left = newNode->right = NULL;
14     return newNode;
15 }
16
17 struct Node* insert(struct Node* root, int data) {
18     if (root == NULL)
19         return createNode(data);
20     if (data < root->data)
21         root->left = insert(root->left, data);

```

INPUT

Your INPUT goes here!

OUTPUT

Original Inorder: 3 5 7 10 12 15 18
Mirrored Inorder: 18 15 12 10 7 5 3

Screenshot 58



SIMATS Online Programming Lab
DS PROGRAMMING


Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed

AVL Tree- Balancing

AVL TREE-BALANCING

Insert the elements: 30, 20, 10 Use C Language to implement the Process. Show the AVL Tree before balancing. Perform the required rotation. Display the balanced AVL Tree. Calculate the balance factor of each node.

PROGRAM EDITOR

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node* left;
7     struct Node* right;
8     int height;
9 };
10
11 int max(int a, int b) {
12     return (a > b) ? a : b;
13 }
14
15 int height(struct Node* n) {
16     if (n == NULL)
17         return 0;
18     return n->height;
19 }
20
21 struct Node* newNode(int data) {

```

Run
Save

C

INPUT

Your INPUT goes here!

OUTPUT

```

AVL Tree Balancing Process (Inserting: 30, 20, 10)

Balanced AVL Tree (Preorder): 20 10 30

Node 10 Balance Factor: 0

```

Screenshot 59



SIMATS Online Programming Lab
DS PROGRAMMING


Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed

AVL Tree -Balance 1

AVL TREE - BALANCE NODE

Create an AVL Tree from user-entered values and perform following tasks. Calculate the height of every node. Determine the balance factor of each node. Display nodes that violate AVL property before balancing.

PROGRAM EDITOR

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node* left;
7     struct Node* right;
8     int height;
9 };
10
11 int max(int a, int b) {
12     return (a > b) ? a : b;
13 }
14
15 int height(struct Node* n) {
16     if (n == NULL)
17         return 0;
18     return n->height;
19 }
20
21

```

Run
Save

C

INPUT

3
30 20 10

OUTPUT

```

Node 10 -> Height: 1, Balance Factor: 0
Node 20 -> Height: 2, Balance Factor: 1
Node 30 -> Height: 3, Balance Factor: 2 [Violates AVL Property!]

```

Screenshot 60



SIMATS Online Programming Lab
DS PROGRAMMING

Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed
AVL Tree- Sequenc
AVL TREE-SEQUENCE

Insert the following sorted sequence: 10, 20, 30, 40, 50, 60, 70. Perform the following operations: 1) Construct a BST. 2) Construct an AVL Tree 3) Compare Height, Number of Rotations.

PROGRAM EDITOR

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node* left;
7     struct Node* right;
8     int height;
9 };
10
11 int max(int a, int b) {
12     return (a > b) ? a : b;
13 }
14
15 int height(struct Node* n) {
16     if (n == NULL)
17         return 0;
18     return n->height;
19 }
20
21 struct Node* newNode(int data) {

```

OUTPUT

```

1) BST Height: 6
2) AVL Tree Height: 2
3) Number of Rotations: 4

```

INPUT

Your INPUT goes here!

Screenshot 61



SIMATS Online Programming Lab
DS PROGRAMMING

Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed
Menu-Driven AVL 1
MENU-DRIVEN AVL TREE

Develop a menu-driven AVL Tree program supporting: Insert, Delete, Search, Display Inorder Traversal, Display Height and Exit.

PROGRAM EDITOR

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node* left;
7     struct Node* right;
8     int height;
9 };
10
11 int max(int a, int b) {
12     return (a > b) ? a : b;
13 }
14
15 int height(struct Node* n) {
16     if (n == NULL)
17         return 0;
18     return n->height;
19 }
20
21 struct Node* newNode(int data) {

```

OUTPUT

```

Inorder Traversal: 30 50 70
Height of AVL Tree: 1

```

INPUT

```

1 50
1 30
1 70
4
5
6

```

Screenshot 62



SIMATS Online Programming Lab
DS PROGRAMMING
Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed
AVL Tree- Impleme
AVL TREE-IMPLEMENT ROTATIONS
Construct an AVL Tree and implement rotations LL, RR, LR, RL and balancing using subprograms and verify it.

PROGRAM EDITOR
C
Run Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node* left;
7     struct Node* right;
8     int height;
9 };
10
11 int max(int a, int b) {
12     return (a > b) ? a : b;
13 }
14
15 int height(struct Node* n) {
16     if (n == NULL)
17         return 0;
18     return n->height;
19 }
20
21 struct Node* newNode(int data) {

```

OUTPUT
Preorder traversal of the constructed AVL tree is
30 20 10 25 40 50

INPUT
Your INPUT goes here!

Screenshot 63



SIMATS Online Programming Lab
DS PROGRAMMING
Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed
AVL Tree - Deletio
AVL TREE - DELETION
Write a C program to delete a node 88 from an AVL Tree and rebalance the tree if necessary. Use the following elements 77, 22, 99, 55, 88, 33, 11, 66, 44.

PROGRAM EDITOR
C
Run Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node* left;
7     struct Node* right;
8     int height;
9 };
10
11 int max(int a, int b) {
12     return (a > b) ? a : b;
13 }
14
15 int height(struct Node* n) {
16     if (n == NULL)
17         return 0;
18     return n->height;
19 }
20
21 struct Node* newNode(int data) {

```

OUTPUT
33 22 11 77 55 44 66 99

INPUT
Your INPUT goes here!

Screenshot 64



SIMATS Online Programming Lab
DS PROGRAMMING


Monica Santamil K
192524315RB

QUESTIONS 10 / 10 completed

AVL Tree- Height

AVL TREE- HEIGHT

Write a C program to find the height of an AVL Tree. Create an AVL Tree 50 30 20 40 70 60. Display height after each insertion.

PROGRAM EDITOR

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node* left;
7     struct Node* right;
8     int height;
9 };
10
11 int max(int a, int b) {
12     return (a > b) ? a : b;
13 }
14
15 int height(struct Node* n) {
16     if (n == NULL)
17         return 0;
18     return n->height;
19 }
20
21

```

Run

Save

INPUT

Your INPUT goes here!

OUTPUT

Height after inserting 50: 0
Height after inserting 30: 1
Height after inserting 20: 1
Height after inserting 40: 2
Height after inserting 70: 2

Screenshot 65



SIMATS Online Programming Lab
DS PROGRAMMING


Monica Santamil K
192524315RB

QUESTIONS 10 / 10 completed

AVL Tree- Balance F

AVL TREE- BALANCE FACTOR

Write a C program to display the balance factor of each node in an AVL Tree.

PROGRAM EDITOR

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node* left;
7     struct Node* right;
8     int height;
9 };
10
11 int max(int a, int b) {
12     return (a > b) ? a : b;
13 }
14
15 int height(struct Node* n) {
16     if (n == NULL)
17         return 0;
18     return n->height;
19 }
20
21 struct Node* newNode(int data) {

```

Run

Save

INPUT

3
30 20 10

OUTPUT

Node 10 Balance Factor: 0
Node 30 Balance Factor: 0
Node 20 Balance Factor: 0

Screenshot 66



SIMATS Online Programming Lab
DS PROGRAMMING

 Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed

AVL Rotations with

AVL ROTATIONS WITH BALANCING

Write a C program to demonstrate all four AVL rotations. LL Rotation, RR Rotation, LR Rotation, RL Rotation. Display AVL Tree before and after balancing.

PROGRAM EDITOR
C
Run Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node* left;
7     struct Node* right;
8     int height;
9 };
10
11 int max(int a, int b) {
12     return (a > b) ? a : b;
13 }
14
15 int height(struct Node* n) {
16     if (n == NULL)
17         return 0;
18     return n->height;
19 }
20
21 struct Node* newNode(int data) {

```

INPUT
3
30 20 10

OUTPUT

```

Tree before balancing (Preorder): 30 20 10
Tree after balancing (Preorder): 20 10 30

```

Screenshot 67

HASHING :



SIMATS Online Programming Lab
DS PROGRAMMING

 Monica Sentamil K
192524315RB

QUESTIONS 5 / 5 completed

Hash-Linear Probing

HASH-LINEAR PROBING

Write a program to create a hash table of size 10, insert the elements 25, 35, 15, 45, 95. If any collisions occurred, then resolve collisions using linear probing. Finally display the hash table after each insertion.

PROGRAM EDITOR
C
Run Save

```

1 #include <stdio.h>
2
3 #define SIZE 10
4
5 void display(int table[]) {
6     for (int i = 0; i < SIZE; i++) {
7         if (table[i] == -1)
8             printf("- ");
9         else
10            printf("%d ", table[i]);
11     }
12     printf("\n");
13 }
14
15 int main() {
16     int table[SIZE];
17     for (int i = 0; i < SIZE; i++) {
18         table[i] = -1;
19     }
20
21     int elements[] = {25, 35, 15, 45, 95};

```

INPUT
Your INPUT goes here!

OUTPUT

```

- - - - 25 - - -
- - - - 25 35 - -
- - - - 25 35 15 -
- - - - 25 35 15 45 -
- - - - 25 35 15 45 95

```

Screenshot 68

SIMATS Online Programming Lab
DS PROGRAMMING

QUESTIONS 5/5 completed

Hashing-Delete

HASHING-DELETE

Write a program to perform the following operations using hashing: insert an element, search an element and delete an element

PROGRAM EDITOR

```

1 #include <stdio.h>
2
3 #define SIZE 10
4
5 void display(int table[]) {
6     for (int i = 0; i < SIZE; i++) {
7         if (table[i] == -1)
8             printf("- ");
9         else
10            printf("%d ", table[i]);
11    }
12    printf("\n");
13 }
14
15 int main() {
16     int table[SIZE];
17     for (int i = 0; i < SIZE; i++) {
18         table[i] = -1;
19     }
20
21     int choice, key, index, found;

```

OUTPUT

```

- - - - - 35 - - -

```

INPUT

```

1 25
2 35
3 25
4
5

```

Screenshot 69

SIMATS Online Programming Lab
DS PROGRAMMING

QUESTIONS 5/5 completed

Hash-Store & Delete

HASH-STORE & DELETE

Develop a program using hashing to store employee details, search employee details using employee ID and delete employee records

PROGRAM EDITOR

```

1 #include <stdio.h>
2 #include <string.h>
3
4 #define SIZE 10
5
6 struct Employee {
7     int id;
8     char name[50];
9     int isOccupied; // 0: empty, 1: occupied, -1: deleted
10 };
11
12 void initTable(struct Employee table[]) {
13     for (int i = 0; i < SIZE; i++) {
14         table[i].id = -1;
15         table[i].isOccupied = 0;
16         strcpy(table[i].name, "");
17     }
18 }
19
20 void insertEmployee(struct Employee table[], int id, const char* name) {
21

```

OUTPUT

```

Index 0: -
Index 1: ID = 101, Name = John
Index 2: ID = 102, Name = Alice
Index 3: -
Index 4: -

```

INPUT

```

1 101 John
2 102 Alice
3
4
5 101
6
7
8
9

```

Screenshot 70

SIMATS Online Programming Lab
DS PROGRAMMING

QUESTIONS 5 / 5 completed
Hash-Store & Delete
HASH-STORE & DELETE
Develop a program using hashing to store employee details, search employee details using employee ID and delete employee records

PROGRAM EDITOR

```

1 #include <stdio.h>
2 #include <string.h>
3
4 #define SIZE 10
5
6 struct Employee {
7     int id;
8     char name[50];
9     int isOccupied; // 0: empty, 1: occupied, -1: deleted
10 };
11
12 void initTable(struct Employee table[]) {
13     for (int i = 0; i < SIZE; i++) {
14         table[i].id = -1;
15         table[i].isOccupied = 0;
16         strcpy(table[i].name, "");
17     }
18 }
19
20 void insertEmployee(struct Employee table[], int id, const char* name) {
21

```

INPUT
1 101 John
1 102 Alice
4
2 101
3 101
4
5

OUTPUT
Index 0: -
Index 1: ID = 101, Name = John
Index 2: ID = 102, Name = Alice
Index 3: -
Index 4: -

Screenshot 71

SIMATS Online Programming Lab
DS PROGRAMMING

QUESTIONS 5 / 5 completed
Quadratic Probing
QUADRATIC PROBING
Write a program to insert the keys 12, 22, 32, 42, 52 into a hash table using quadratic probing and display the final hash table.

PROGRAM EDITOR

```

1 #include <stdio.h>
2
3 #define SIZE 10
4
5 void display(int table[]) {
6     for (int i = 0; i < SIZE; i++) {
7         if (table[i] == -1)
8             printf("- ");
9         else
10            printf("%d ", table[i]);
11     }
12     printf("\n");
13 }
14
15 int main() {
16     int table[SIZE];
17     for (int i = 0; i < SIZE; i++) {
18         table[i] = -1;
19     }
20
21     int keys[] = {12, 22, 32, 42, 52};

```

INPUT
Your INPUT goes here!

OUTPUT
- 42 12 22 - - 32 - 52 -

Screenshot 72

PRIORITY QUEUE AND BINARY HEAPS:

SIMATS
ENGINEERING

SIMATS Online Programming Lab
DS PROGRAMMING

Monica Sentamil K
192524315R8

QUESTIONS 10 / 10 completed
 Max Heap
MAX HEAP
 "Write a C program to perform the following operations:
 Insert the employee salaries: 45000, 52000, 61000, 48000, 75000, 68000, 59000. Display the Max Heap after every insertion
 Delete the highest salary from the heap. Display the heap after deletion"

PROGRAM EDITOR

C

Run Save

```

1 #include <stdio.h>
2
3 void swap(int *a, int *b) {
4     int temp = *a;
5     *a = *b;
6     *b = temp;
7 }
8
9 void heapifyUp(int heap[], int index) {
10     int parent = (index - 1) / 2;
11     if (index > 0 && heap[index] > heap[parent]) {
12         swap(&heap[index], &heap[parent]);
13         heapifyUp(heap, parent);
14     }
15 }
16
17 void heapifyDown(int heap[], int size, int index) {
18     int largest = index;
19     int left = 2 * index + 1;
20     int right = 2 * index + 2;
21

```

OUTPUT
 Max Heap after every insertion:
 45000
 52000 45000
 61000 45000 52000
 61000 48000 52000 45000
 68000 45000 52000 48000 75000

INPUT
 Your INPUT goes here!

Screenshot 73

SIMATS
ENGINEERING

SIMATS Online Programming Lab
DS PROGRAMMING

Monica Sentamil K
192524315R8

QUESTIONS 10 / 10 completed
 Final Priority Queue
FINAL PRIORITY QUEUE
 "Write a C program to implement the following operations:
 Insert student requests with priorities: (101, 3), (102, 1), (103, 5), (104, 2), (105, 4)
 Display the queue elements in priority order
 Delete the highest priority request
 Insert a new request (106, 6)
 Display the final

PROGRAM EDITOR

C

Run Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Request {
5     int id;
6     int priority;
7 };
8
9 void swap(struct Request *a, struct Request *b) {
10     struct Request temp = *a;
11     *a = *b;
12     *b = temp;
13 }
14
15 void heapifyUp(struct Request heap[], int index) {
16     int parent = (index - 1) / 2;
17     if (index > 0 && heap[index].priority > heap[parent].priority) {
18         swap(&heap[index], &heap[parent]);
19         heapifyUp(heap, parent);
20     }
21 }

```

OUTPUT
 Queue elements in priority order (Max Heap):
 ID: 103, Priority: 5
 ID: 105, Priority: 4
 ID: 101, Priority: 3
 ID: 102, Priority: 1

INPUT
 Your INPUT goes here!

Screenshot 74



SIMATS Online Programming Lab
DS PROGRAMMING


Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed

Heap Sort

HEAP SORT

"Write a C program to implement Heap Sort for the following set of numbers:
45, 12, 78, 34, 23, 89, 67, 10, 56, 99
The program should:
Construct the Max Heap
Display the heap after each adjustment
Sort the elements in ascending order using Heap Sort
Display the sorted

PROGRAM EDITOR

```

1 #include <stdio.h>
2
3 void swap(int *a, int *b) {
4     int temp = *a;
5     *a = *b;
6     *b = temp;
7 }
8
9 void heapify(int arr[], int n, int i) {
10    int largest = i;
11    int left = 2 * i + 1;
12    int right = 2 * i + 2;
13
14    if (left < n && arr[left] > arr[largest])
15        largest = left;
16
17    if (right < n && arr[right] > arr[largest])
18        largest = right;
19
20    if (largest != i) {
21        swap(&arr[i], &arr[largest]);

```

Run

Save

INPUT

Your INPUT goes here!

OUTPUT

Max Heap constructed: 99 56 89 45 23 78 67 10 34 12
Heap after adjustment: 89 56 78 45 23 12 67 10 34 99
Heap after adjustment: 78 56 67 45 23 12 34 10 89 99
Heap after adjustment: 67 56 34 45 23 12 10 78 89 99
Heap after adjustment: 56 45 34 10 23 12 67 78 89 99

Screenshot 75



SIMATS Online Programming Lab
DS PROGRAMMING


Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed

Heap and Priority Queue

HEAP AND PRIORITY QUEUE

Write a C program to implement all Heap and Priority Queue operations through a menu-driven program."

PROGRAM EDITOR

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define MAX_SIZE 100
5
6 int heap[MAX_SIZE];
7 int size = 0;
8
9 void swap(int *a, int *b) {
10    int temp = *a;
11    *a = *b;
12    *b = temp;
13 }
14
15 void heapifyUp(int index) {
16    int parent = (index - 1) / 2;
17    if (index > 0 && heap[index] > heap[parent]) {
18        swap(&heap[index], &heap[parent]);
19        heapifyUp(parent);
20    }
21 }

```

Run

Save

INPUT

1 40
1 30
1 20
3
4

OUTPUT

40 30 20

Screenshot 76

SIMATS Online Programming Lab
DS PROGRAMMING

Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed

Heap Property

HEAP PROPERTY

Write a C program to check whether a given binary tree satisfies Heap property.

PROGRAM EDITOR

C

Run

Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <stdbool.h>
4
5 struct Node {
6     int data;
7     struct Node* left;
8     struct Node* right;
9 };
10
11 struct Node* newNode(int data) {
12     struct Node* node = (struct Node*)malloc(sizeof(struct Node));
13     node->data = data;
14     node->left = NULL;
15     node->right = NULL;
16     return node;
17 }
18
19 // Function to count the number of nodes in the binary tree
20 int countNodes(struct Node* root) {
21     if (root == NULL)

```

OUTPUT

The given binary tree satisfies the Heap property.

INPUT

Your INPUT goes here!

Screenshot 77

SIMATS Online Programming Lab
DS PROGRAMMING

Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed

Heap Operations

HEAP OPERATIONS

"Write a menu-driven C program to implement all Heap operations for the following elements: 18, 25, 40, 12, 30, 50, 10, 45 and perform the following operations: Heap creation, Insertion, Deletion, Heap display and Searching an element"

PROGRAM EDITOR

C

Run

Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define MAX_SIZE 100
5
6 int heap[MAX_SIZE];
7 int size = 0;
8
9 void swap(int *a, int *b) {
10     int temp = *a;
11     *a = *b;
12     *b = temp;
13 }
14
15 void heapifyUp(int index) {
16     int parent = (index - 1) / 2;
17     if (index > 0 && heap[index] > heap[parent]) {
18         swap(&heap[index], &heap[parent]);
19         heapifyUp(parent);
20     }
21 }

```

OUTPUT

50 45 40 30 18 25 10 12
Element 30 found at index 3

INPUT

4
3 30
5

Screenshot 78



SIMATS Online Programming Lab

DS PROGRAMMING

Monica Sentamil K
192524315R8

QUESTIONS 10 / 10 completed

Merging Heaps

MERGING HEAPS

Write a C program to merge two Max Heaps into a single Heap.

PROGRAM EDITOR

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 void swap(int *a, int *b) {
5     int temp = *a;
6     *a = *b;
7     *b = temp;
8 }
9
10 void heapify(int arr[], int n, int i) {
11     int largest = i;
12     int left = 2 * i + 1;
13     int right = 2 * i + 2;
14
15     if (left < n && arr[left] > arr[largest])
16         largest = left;
17
18     if (right < n && arr[right] > arr[largest])
19         largest = right;
20
21     if (largest != i) {

```

Run

Save

INPUT

```

3
10 5 2
3
20 15 10

```

OUTPUT

```

20 15 10 5 10 2

```

Screenshot 79



SIMATS Online Programming Lab

DS PROGRAMMING

Monica Sentamil K
192524315R8

QUESTIONS 10 / 10 completed

Priority Queue

PRIORITY QUEUE

"Write a C program to perform the following operations using priority queue: Insert patient records with severity levels: Ravi-4, Kumar-2, Priya-5, Anu-1, Deepak-3 Display the patients in priority order Treat and remove the highest priority patient Add a new patient "Arun-6"

PROGRAM EDITOR

```

1 #include <stdio.h>
2 #include <string.h>
3
4 struct Patient {
5     char name[50];
6     int severity;
7 };
8
9 void swap(struct Patient *a, struct Patient *b) {
10     struct Patient temp = *a;
11     *a = *b;
12     *b = temp;
13 }
14
15 void heapifyUp(struct Patient heap[], int index) {
16     int parent = (index - 1) / 2;
17     if (index > 0 && heap[index].severity > heap[parent].severity) {
18         swap(&heap[index], &heap[parent]);
19         heapifyUp(heap, parent);
20     }
21 }

```

Run

Save

INPUT

Your INPUT goes here!

OUTPUT

```

Priya-5
Deepak-3
Ravi-4
Anu-1
Kumar-2

```

Screenshot 80



SIMATS Online Programming Lab
DS PROGRAMMING

Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed

Heap- Min & Max

HEAP- MIN & MAX ELEMENT

Write a C program to find the maximum and minimum element using Heap.

PROGRAM EDITOR

C

Run Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 void swap(int *a, int *b) {
5     int temp = *a;
6     *a = *b;
7     *b = temp;
8 }
9
10 void heapify(int arr[], int n, int i) {
11     int largest = i;
12     int left = 2 * i + 1;
13     int right = 2 * i + 2;
14
15     if (left < n && arr[left] > arr[largest])
16         largest = left;
17
18     if (right < n && arr[right] > arr[largest])
19         largest = right;
20
21     if (largest != i) {

```

INPUT

5
12 45 7 23 56

OUTPUT

Maximum element: 56
Minimum element: 7

Screenshot 81



SIMATS Online Programming Lab
DS PROGRAMMING

Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed

Min Heap- Operati

MIN HEAP- OPERATIONS

"Write a C program to construct a Min Heap using the following data: 55, 20, 35, 10, 90, 40, 15, 25 and perform the following operations: Insert elements one by one into the Min Heap. Display the heap after every insertion. Delete the minimum element. Display the final heap structure"

PROGRAM EDITOR

C

Run Save

```

1 #include <stdio.h>
2
3 void swap(int *a, int *b) {
4     int temp = *a;
5     *a = *b;
6     *b = temp;
7 }
8
9 void heapifyUp(int heap[], int index) {
10     int parent = (index - 1) / 2;
11     if (index > 0 && heap[index] < heap[parent]) {
12         swap(&heap[index], &heap[parent]);
13         heapifyUp(heap, parent);
14     }
15 }
16
17 void heapifyDown(int heap[], int size, int index) {
18     int smallest = index;
19     int left = 2 * index + 1;
20     int right = 2 * index + 2;
21

```

INPUT

Your INPUT goes here!

OUTPUT

55
20 55
20 55 35
10 20 35 55
10 20 35 55 90
10 20 35 55 90 40

Screenshot 82

SORTING AND TIME COMPLEXITY:



SIMATS Online Programming Lab
DS PROGRAMMING

Monica Sentamil K
192524315RB

QUESTIONS 5 / 5 completed

Quick Sort

QUICK SORT

Write a program to sort the array 50, 23, 9, 18, 61, 32 using Quick Sort.

PROGRAM EDITOR

C

Run Save

```
1 #include <stdio.h>
2
3 void swap(int* a, int* b) {
4     int t = *a;
5     *a = *b;
6     *b = t;
7 }
8
9 int partition(int arr[], int low, int high) {
10     int pivot = arr[high];
11     int i = (low - 1);
12
13     for (int j = low; j <= high - 1; j++) {
14         if (arr[j] < pivot) {
15             i++;
16             swap(&arr[i], &arr[j]);
17         }
18     }
19     swap(&arr[i + 1], &arr[high]);
20     return (i + 1);
21 }
```

OUTPUT

9 18 23 32 50 61

INPUT

Your INPUT goes here!

Screenshot 83



SIMATS Online Programming Lab
DS PROGRAMMING

Monica Sentamil K
192524315RB

QUESTIONS 5 / 5 completed

Bubble Sort

BUBBLE SORT

"Write a program that implements the bubble sort"

PROGRAM EDITOR

C

Run Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 void bubbleSort(int arr[], int n) {
5     for (int i = 0; i < n - 1; i++) {
6         for (int j = 0; j < n - i - 1; j++) {
7             if (arr[j] > arr[j + 1]) {
8                 int temp = arr[j];
9                 arr[j] = arr[j + 1];
10                arr[j + 1] = temp;
11            }
12        }
13    }
14 }
15
16 int main() {
17     int n;
18     if (scanf("%d", &n) != 1 || n <= 0) return 0;
19
20     int *arr = (int *)malloc(n * sizeof(int));
21     if (arr == NULL) return 0;
```

OUTPUT

12 22 25 34 64

INPUT

5
64 34 25 12 22

Screenshot 84



SIMATS Online Programming Lab
DS PROGRAMMING

 Monica Sentamil K
192524315RB

QUESTIONS 5 / 5 completed

Insertion Sort

INSERTION SORT
Write a program sort the elements 9, 5, 1, 4, 3 using Insertion Sort.

PROGRAM EDITOR

C

Run Save

```

1 #include <stdio.h>
2
3 void insertionSort(int arr[], int n) {
4     for (int i = 1; i < n; i++) {
5         int key = arr[i];
6         int j = i - 1;
7
8         while (j >= 0 && arr[j] > key) {
9             arr[j + 1] = arr[j];
10            j--;
11        }
12        arr[j + 1] = key;
13    }
14 }
15
16 int main() {
17     int arr[] = {9, 5, 1, 4, 3};
18     int n = sizeof(arr) / sizeof(arr[0]);
19     insertionSort(arr, n);
20 }

```

OUTPUT

```

1 3 4 5 9

```

INPUT

Your INPUT goes here!

Screenshot 85



SIMATS Online Programming Lab
DS PROGRAMMING

 Monica Sentamil K
192524315RB

QUESTIONS 5 / 5 completed

Merge Sort

MERGE SORT
Apply Merge Sort to the array 38, 27, 43, 3, 9, 82, 10.

PROGRAM EDITOR

C

Run Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 void merge(int arr[], int left, int mid, int right) {
5     int n1 = mid - left + 1;
6     int n2 = right - mid;
7
8     int *L = (int *)malloc(n1 * sizeof(int));
9     int *R = (int *)malloc(n2 * sizeof(int));
10
11     for (int i = 0; i < n1; i++)
12         L[i] = arr[left + i];
13     for (int j = 0; j < n2; j++)
14         R[j] = arr[mid + 1 + j];
15
16     int i = 0, j = 0, k = left;
17     while (i < n1 && j < n2) {
18         if (L[i] <= R[j]) {
19             arr[k] = L[i];
20             i++;
21         } else {

```

OUTPUT

```

3 9 10 27 38 43 82

```

INPUT

Your INPUT goes here!

Screenshot 86



SIMATS Online Programming Lab
DS PROGRAMMING

 Monica Sentamil K
192524315RB

QUESTIONS 5 / 5 completed

Heap Sort

HEAP SORT

Write a program that implements the Heap sort for the following list of elements :
23,56,12,34,78 and 29

PROGRAM EDITOR

C

Run

Save

```

1 #include <stdio.h>
2
3 void swap(int *a, int *b) {
4     int temp = *a;
5     *a = *b;
6     *b = temp;
7 }
8
9 void heapify(int arr[], int n, int i) {
10     int largest = i;
11     int left = 2 * i + 1;
12     int right = 2 * i + 2;
13
14     if (left < n && arr[left] > arr[largest])
15         largest = left;
16
17     if (right < n && arr[right] > arr[largest])
18         largest = right;
19
20     if (largest != i) {
21         swap(&arr[i], &arr[largest]);

```

OUTPUT

12 23 29 34 56 78

INPUT

Your INPUT goes here!

Screenshot 87

SPANNING TREE:



SIMATS Online Programming Lab
DS PROGRAMMING

 Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed

Min Spanning Tree

MIN SPANNING TREE

"Write a C program to construct the Minimum Spanning Tree of a weighted undirected graph using Prim's Algorithm.

Input Matrix:

0 2 0 6 0
2 0 3 8 5
0 3 0 0 7
6 8 0 0 9
0 5 7 9 0"

PROGRAM EDITOR

C

Run

Save

```

1 #include <stdio.h>
2 #include <stdbool.h>
3
4 #define V 5
5
6 int minKey(int key[], bool mstSet[]) {
7     int min = 999999, min_index;
8
9     for (int v = 0; v < V; v++)
10         if (mstSet[v] == false && key[v] < min)
11             min = key[v], min_index = v;
12
13     return min_index;
14 }
15
16 void printMST(int parent[], int graph[V][V]) {
17     printf("Edge \tWeight\n");
18     for (int i = 1; i < V; i++)
19         printf("%d - %d \t%d \n", parent[i], i, graph[i][parent[i]]);
20 }
21

```

OUTPUT

Edge Weight
0 - 1 2
1 - 2 3
0 - 3 6
1 - 4 5

INPUT

Your INPUT goes here!

Screenshot 88



SIMATS Online Programming Lab
DS PROGRAMMING

 Monica Sentamil K
192524315R8

QUESTIONS 10 / 10 completed

Span Tree- Kruskal'

SPAN TREE-KRUSKAL'S ALGORITHM

"Write a C program to find the Minimum Spanning Tree using Kruskal's Algorithm."

Input Edges:


```

0 1 4
0 2 2
1 2 1
1 3 5
2 3 8
2 4 10
3 4 2

```

PROGRAM EDITOR

```

1 #include <stdio.h>
2 #include <stdbool.h>
3
4 #define V 5
5
6 int minKey(int key[], bool mstSet[]) {
7     int min = 999999, min_index;
8
9     for (int v = 0; v < V; v++)
10         if (mstSet[v] == false && key[v] < min)
11             min = key[v], min_index = v;
12
13     return min_index;
14 }
15
16 void printMST(int parent[], int graph[V][V]) {
17     printf("Edge \tWeight\n");
18     for (int i = 1; i < V; i++)
19         printf("%d - %d \t%d \n", parent[i], i, graph[i][parent[i]]);
20 }
21

```

Run

Save

INPUT

Your INPUT goes here!

OUTPUT

```

Edge      Weight
0 - 1      2
1 - 2      3
0 - 3      6
1 - 4      5

```

Screenshot 89



SIMATS Online Programming Lab
DS PROGRAMMING

 Monica Sentamil K
192524315R8

QUESTIONS 10 / 10 completed

MST using Prim's A

MST USING PRIM'S ALGORITHM

"Construct the MST using Prim's Algorithm and display its total weight."

Input Matrix:


```

0 4 3 0
4 0 2 5
3 2 0 7
0 5 7 0

```

PROGRAM EDITOR

```

1 #include <stdio.h>
2 #include <stdbool.h>
3
4 #define V 4
5
6 int minKey(int key[], bool mstSet[]) {
7     int min = 999999, min_index;
8
9     for (int v = 0; v < V; v++)
10         if (mstSet[v] == false && key[v] < min)
11             min = key[v], min_index = v;
12
13     return min_index;
14 }
15
16 void primMST(int graph[V][V]) {
17     int parent[V];
18     int key[V];
19     bool mstSet[V];
20
21     for (int i = 0; i < V; i++)

```

Run

Save

INPUT

Your INPUT goes here!

OUTPUT

```

Total Weight: 10

```

Screenshot 90



SIMATS Online Programming Lab
DS PROGRAMMING



QUESTIONS 10 / 10 completed

Building- Min Cabl

BUILDING- MIN CABLE LENGTH

"Buildings in a campus are represented as vertices. Determine the minimum cable length required to connect all buildings.

Input:

```

0 1 3
0 2 1
1 2 7
1 3 5
2 3 2"
```

PROGRAM EDITOR

C
Run
Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Edge {
5     int src, dest, weight;
6 };
7
8 struct Graph {
9     int V, E;
10    struct Edge* edge;
11 };
12
13 struct Subset {
14     int parent;
15     int rank;
16 };
17
18 struct Graph* createGraph(int V, int E) {
19     struct Graph* graph = (struct Graph*)malloc(sizeof(struct Graph));
20     graph->V = V;
21     graph->E = E;
```

OUTPUT

```

Minimum cable length required: 6
```

INPUT

Your INPUT goes here!

Screenshot 91



SIMATS Online Programming Lab
DS PROGRAMMING



QUESTIONS 10 / 10 completed

MST-Selected Edges

MST-SELECTED EDGES

"Construct the MST and list the selected edges in increasing order of weight.

Input:

```

0 1 2
0 2 6
1 2 3
1 3 8
2 3 5"
```

PROGRAM EDITOR

C
Run
Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Edge {
5     int src, dest, weight;
6 };
7
8 struct Graph {
9     int V, E;
10    struct Edge* edge;
11 };
12
13 struct Subset {
14     int parent;
15     int rank;
16 };
17
18 struct Graph* createGraph(int V, int E) {
19     struct Graph* graph = (struct Graph*)malloc(sizeof(struct Graph));
20     graph->V = V;
21     graph->E = E;
```

OUTPUT

```

0 1 2
1 2 3
2 3 5
```

INPUT

Your INPUT goes here!

Screenshot 92



SIMATS Online Programming Lab
DS PROGRAMMING

 Monica Sentamil K
192524315R8

QUESTIONS 10 / 10 completed

MST- Identifying E

MST- IDENTIFYING EDGES
"Construct the MST and identify the edges that are not included."

Input:

0 1 1
0 2 4
1 2 2
1 3 5
2 3 3

PROGRAM EDITOR

C
Run Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Edge {
5     int src, dest, weight;
6 };
7
8 struct Graph {
9     int V, E;
10    struct Edge* edge;
11 };
12
13 struct Subset {
14     int parent;
15     int rank;
16 };
17
18 struct Graph* createGraph(int V, int E) {
19     struct Graph* graph = (struct Graph*)malloc(sizeof(struct Graph));
20     graph->V = V;

```

OUTPUT

0 2 4
1 3 5

INPUT

Your INPUT goes here!

Screenshot 93



SIMATS Online Programming Lab
DS PROGRAMMING

 Monica Sentamil K
192524315R8

QUESTIONS 10 / 10 completed

Adjacency Matrix

ADJACENCY MATRIX
"Implement Prim's Algorithm using an adjacency matrix representation."

Input Matrix:

0 1 4 0 0
1 0 2 7 0
4 2 0 3 5
0 7 3 0 6
0 0 5 6 0

PROGRAM EDITOR

C
Run Save

```

1 #include <stdio.h>
2 #include <stdbool.h>
3
4 #define V 5
5
6 int minKey(int key[], bool mstSet[]) {
7     int min = 999999, min_index;
8
9     for (int v = 0; v < V; v++)
10         if (mstSet[v] == false && key[v] < min)
11             min = key[v], min_index = v;
12
13     return min_index;
14 }
15
16 void printMST(int parent[], int graph[V][V]) {
17     for (int i = 1; i < V; i++)
18         printf("%d - %d\t%d\n", parent[i], i, graph[i][parent[i]]);
19 }
20
21 void printMST(int graph[V][V]) {

```

OUTPUT

0 - 1 1
1 - 2 2
2 - 3 3
2 - 4 5

INPUT

Your INPUT goes here!

Screenshot 94



SIMATS Online Programming Lab
DS PROGRAMMING

Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed

MST- Savings

MST- SAVINGS

"A network originally uses all edges. Determine the savings achieved by using only the MST.

Input:

```

0 1 4
0 2 8
1 2 2
1 3 6
2 3 3"
```

PROGRAM EDITOR

C
Run
Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Edge {
5     int src, dest, weight;
6 };
7
8 struct Graph {
9     int V, E;
10    struct Edge* edge;
11 };
12
13 struct Subset {
14     int parent;
15     int rank;
16 };
17
18 struct Graph* createGraph(int V, int E) {
19     struct Graph* graph = (struct Graph*)malloc(sizeof(struct Graph));
20     graph->V = V;
21 }
```

OUTPUT

```

Error: Execution timeout (>5 seconds) - possible infinite loop
```

INPUT

Your INPUT goes here!

Screenshot 95



SIMATS Online Programming Lab
DS PROGRAMMING

Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed

MST-Kruskal's Alg

MST-KRUSKAL'S ALGORITHM

Implement Kruskal's Algorithm and display the edges after sorting by weight.

PROGRAM EDITOR

C
Run
Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Edge {
5     int src, dest, weight;
6 };
7
8 struct Graph {
9     int V, E;
10    struct Edge* edge;
11 };
12
13 struct Subset {
14     int parent;
15     int rank;
16 };
17
18 struct Graph* createGraph(int V, int E) {
19     struct Graph* graph = (struct Graph*)malloc(sizeof(struct Graph));
20     graph->V = V;
21 }
```

OUTPUT

```

0 1 2
1 2 3
2 3 5
0 2 6
1 3 8
```

INPUT

```

4 5
0 1 2
0 2 6
1 2 3
1 3 8
2 3 5
```

Screenshot 96

SIMATS
ENGINEERING

SIMATS Online Programming Lab
DS PROGRAMMING

Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed
 MST- Edges Reject
MST- EDGES REJECTION
 "Display the number of edges rejected because they form cycles."
 Input:
 0 1 1
 0 2 4
 1 2 2
 1 3 5
 2 3 3"

PROGRAM EDITOR C Run Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Edge {
5     int src, dest, weight;
6 };
7
8 struct Graph {
9     int V, E;
10    struct Edge* edge;
11 };
12
13 struct Subset {
14     int parent;
15     int rank;
16 };
17
18 struct Graph* createGraph(int V, int E) {
19     struct Graph* graph = (struct Graph*)malloc(sizeof(struct Graph));
20     graph->V = V;
    
```

OUTPUT
 0

INPUT
 Your INPUT goes here!

Screenshot 97

GRAPH TRAVERSAL:

SIMATS
ENGINEERING

SIMATS Online Programming Lab
DS PROGRAMMING

Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed
 Breadth First Search
BREADTH FIRST SEARCH TRAVERSAL
 Write a C program to represent a graph using an adjacency matrix and perform Breadth First Search (BFS) traversal starting from a user-specified vertex. Display the order in which the vertices are visited during the traversal.

PROGRAM EDITOR C Run Save

```

1 #include <stdio.h>
2
3 #define MAX 100
4
5 void bfs(int adj[MAX][MAX], int n, int startVertex) {
6     int visited[MAX] = {0};
7     int queue[MAX];
8     int front = 0, rear = 0;
9
10    visited[startVertex] = 1;
11    queue[rear++] = startVertex;
12
13    while (front < rear) {
14        int currentVertex = queue[front++];
15        printf("%d ", currentVertex);
16
17        for (int i = 0; i < n; i++) {
18            if (adj[currentVertex][i] == 1 && !visited[i]) {
19                visited[i] = 1;
20                queue[rear++] = i;
21            }
22        }
23    }
24 }
    
```

OUTPUT
 0 1 2 3

INPUT
 4
 0 1 1 0
 1 0 0 1
 1 0 0 1
 0 1 1 0
 0

Screenshot 98



SIMATS Online Programming Lab
DS PROGRAMMING

 Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed
Depth First Search
DEPTH FIRST SEARCH TRAVERSAL

Develop a C program to represent a graph using an adjacency matrix and perform Depth First Search (DFS) traversal starting from a given source vertex. Print the sequence of visited vertices and explain how DFS explores the graph.

PROGRAM EDITOR C Run Save

```

1 #include <stdio.h>
2
3 #define MAX 100
4
5 void dfs(int adj[MAX][MAX], int n, int vertex, int visited[MAX]) {
6     printf("%d ", vertex);
7     visited[vertex] = 1;
8
9     for (int i = 0; i < n; i++) {
10         if (adj[vertex][i] == 1 && !visited[i]) {
11             dfs(adj, n, i, visited);
12         }
13     }
14 }
15
16 int main() {
17     int n, startVertex;
18
19     if (scanf("%d", &n) != 1) return 0;
20
21     int adj[MAX][MAX];

```

OUTPUT

```
0 1 3 2
```

INPUT

```

4
0 1 1 0
1 0 0 1
1 0 0 1
0 1 1 0
0

```

Screenshot 99



SIMATS Online Programming Lab
DS PROGRAMMING

 Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed
DFS- without Recu
DFS- WITHOUT RECURSION

Write a program to implement Depth First Search traversal without recursion by using an explicit stack data structure. Compare the output with the recursive DFS traversal for the same graph.

PROGRAM EDITOR C Run Save

```

1 #include <stdio.h>
2
3 #define MAX 100
4
5 void dfsIterative(int adj[MAX][MAX], int n, int startVertex) {
6     int visited[MAX] = {0};
7     int stack[MAX];
8     int top = -1;
9
10    // Push the starting vertex onto the stack
11    stack[++top] = startVertex;
12
13    while (top >= 0) {
14        int currentVertex = stack[top--];
15
16        if (!visited[currentVertex]) {
17            printf("%d ", currentVertex);
18            visited[currentVertex] = 1;
19        }
20
21        // To match standard recursive DFS traversal order (visiting left to right)

```

OUTPUT

```
0 1 3 2
```

INPUT

```

4
0 1 1 0
1 0 0 1
1 0 0 1
0 1 1 0
0

```

Screenshot 100

SIMATS Online Programming Lab
DS PROGRAMMING

Monica Santamil K
192524315RB

QUESTIONS 10 / 10 completed

Graph- Adjacency |

GRAPH-ADJACENCY LIST

Create a graph using adjacency list representation and implement Breadth First Search traversal. Demonstrate how adjacency lists reduce memory usage for sparse graphs compared to adjacency matrices.

PROGRAM EDITOR
C
Run Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 // Structure for a node in the adjacency list
5 struct Node {
6     int vertex;
7     struct Node* next;
8 };
9
10 // Structure for the adjacency list
11 struct Graph {
12     int numVertices;
13     struct Node** adjlists;
14 };
15
16 // Create a node
17 struct Node* createNode(int v) {
18     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
19     newNode->vertex = v;
20     newNode->next = NULL;
21     return newNode;

```

INPUT

```

4 4
0 1
0 2
1 3
2 3
0

```

OUTPUT

```

0 2 1 3

```

Screenshot 101

SIMATS Online Programming Lab
DS PROGRAMMING

Monica Santamil K
192524315RB

QUESTIONS 10 / 10 completed

GT- Connection- Tr

GT-CONNECTION-TRAVERSAL SEQ

Write a program that uses graph traversal techniques to determine whether a given undirected graph is connected or disconnected. Display an appropriate message along with the traversal sequence.

PROGRAM EDITOR
C
Run Save

```

1 #include <stdio.h>
2
3 #define MAX 100
4
5 void checkConnection(int adj[MAX][MAX], int n) {
6     int visited[MAX] = {0};
7     int queue[MAX];
8     int front = 0, rear = 0;
9
10    // Start traversal from vertex 0
11    visited[0] = 1;
12    queue[rear++] = 0;
13
14    while (front < rear) {
15        int currentVertex = queue[front++];
16
17        for (int i = 0; i < n; i++) {
18            if (adj[currentVertex][i] == 1 && !visited[i]) {
19                visited[i] = 1;
20                queue[rear++] = i;
21            }
22        }
23    }

```

INPUT

```

4
0 1 1 0
1 0 0 1
1 0 0 1
0 1 1 0

```

OUTPUT

```

Traversal sequence: 0 1 2 3
The graph is connected.

```

Screenshot 102



SIMATS Online Programming Lab
DS PROGRAMMING



QUESTIONS 10 / 10 completed

DFS - Vertex

DFS - VERTEX

Develop a program to determine whether a path exists between a specified source vertex and destination vertex using DFS traversal. If a path exists, display the vertices visited while searching for the destination.

PROGRAM EDITOR

C

Run

Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define MAX 100
5
6 int adj[MAX][MAX];
7 int visited[MAX];
8 int path[MAX];
9 int path_index = 0;
10 int found = 0;
11
12 void dfs(int current, int dest, int n) {
13     visited[current] = 1;
14     path[path_index++] = current;
15
16     if (current == dest) {
17         found = 1;
18         return;
19     }
20
21     for (int i = 0; i < n; i++) {

```

OUTPUT

```

Enter the number of vertices: Enter the number of edges: Enter the edges (u v):
Enter source vertex: Enter destination vertex: Path exists between 0 and 3.
Vertices visited: 0 1 2 3

```

INPUT

```

4
3
0 1
1 2
2 3
0
3

```

Screenshot 103



SIMATS Online Programming Lab
DS PROGRAMMING



QUESTIONS 10 / 10 completed

GT- Presence of Cy

GT- PRESENCE OF CYCLES

Implement a graph traversal algorithm to detect the presence of cycles in an undirected graph. The program should report whether the graph contains a cycle and explain the traversal process used.

PROGRAM EDITOR

C

Run

Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define MAX 100
5
6 int adj[MAX][MAX];
7 int visited[MAX];
8 int parent[MAX];
9 int cycle_found = 0;
10
11 int isCyclic(int v, int n) {
12     visited[v] = 1;
13
14     for (int i = 0; i < n; i++) {
15         if (adj[v][i]) {
16             if (!visited[i]) {
17                 parent[i] = v;
18                 if (isCyclic(i, n))
19                     return 1;
20             } else if (parent[v] != i) {

```

OUTPUT

```

Enter the number of vertices: Enter the number of edges: Enter the edges (u v):
Graph contains cycle
Traversal process: Visited an already visited vertex whose parent is not the
current vertex, confirming a cycle.

```

INPUT

```

3
3
0 1
1 2
2 0

```

Screenshot 104



SIMATS Online Programming Lab

DS PROGRAMMING

Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed

GT- Count & Trave

GT- COUNT & TRAVERSE

Write a C program that performs graph traversal from a specified source vertex and counts the total number of vertices that are reachable from that source. Display both the traversal order and the count.

PROGRAM EDITOR

C

Run Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define MAX 100
5
6 int adj[MAX][MAX];
7 int visited[MAX];
8 int traversal[MAX];
9 int count = 0;
10
11 void dfs(int v, int n) {
12     visited[v] = 1;
13     traversal[count++] = v;
14
15     for (int i = 0; i < n; i++) {
16         if (adj[v][i] && !visited[i]) {
17             dfs(i, n);
18         }
19     }
20 }
21

```

OUTPUT

```

Enter the number of vertices: Enter the number of edges: Enter the edges (u v):
Enter the source vertex: Traversal order: 0 1 2 3
Total number of reachable vertices: 4

```

INPUT

```

4
3
0 1
1 2
2 3
0

```

Screenshot 105



SIMATS Online Programming Lab

DS PROGRAMMING

Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed

Graph Creation- Ur

GRAPH CREATION- UNTIL EXIT

Design a menu-driven program that allows the user to create a graph and perform either BFS or DFS traversal from a chosen starting vertex. The program should repeatedly accept user choices until the exit option is selected.

PROGRAM EDITOR

C

Run Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define MAX 100
5
6 int adj[MAX][MAX];
7 int visited[MAX];
8
9 void bfs(int start, int n) {
10     int queue[MAX], front = 0, rear = 0;
11     for (int i = 0; i < n; i++) visited[i] = 0;
12
13     visited[start] = 1;
14     queue[rear++] = start;
15
16     printf("BFS Traversal: ");
17     while (front < rear) {
18         int current = queue[front++];
19         printf("%d ", current);
20
21         for (int i = 0; i < n; i++) {

```

OUTPUT

```

Graph created successfully.
BFS Traversal: 0 1 2 3

```

INPUT

```

1
4 3
0 1
1 2
2 3
2
0
4

```

Screenshot 106



SIMATS Online Programming Lab
DS PROGRAMMING



QUESTIONS 10 / 10 completed
BFS- Level Determi
BFS- LEVEL DETERMINATION
Implement BFS traversal to determine the level (distance from source vertex) of every vertex in a graph and display the results.

PROGRAM EDITOR

C
Run
Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define MAX 100
5
6 int adj[MAX][MAX];
7 int visited[MAX];
8 int level[MAX];
9
10 void bfs(int start, int n) {
11     int queue[MAX], front = 0, rear = 0;
12
13     for (int i = 0; i < n; i++) {
14         visited[i] = 0;
15         level[i] = -1;
16     }
17
18     visited[start] = 1;
19     level[start] = 0;
20     queue[rear++] = start;
21

```

OUTPUT

```

Enter the number of vertices: Enter the number of edges: Enter the edges (u v):
Enter the source vertex: Vertex Level from Source (0)
0      0
1      1
2      2
3      3

```

INPUT

```

4
3
0 1
1 2
2 3
0

```

Screenshot 107

BINARY SEARCH TREE:



SIMATS Online Programming Lab
DS PROGRAMMING



QUESTIONS 10 / 10 completed
Binary Search Tree
BINARY SEARCH TREE
Write a C Program to create a Binary Search Tree by inserting the following elements and traverse it. 77, 33, 11, 99, 66, 55, 1, 2, 8, 88. Perform inorder, Preorder and Post Order.

PROGRAM EDITOR

C
Run
Save

```

1 #include <stdio.h>
2
3
4 struct Node {
5     int data;
6     struct Node *left, *right;
7 };
8
9 struct Node* createNode(int value) {
10     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
11     newNode->data = value;
12     newNode->left = newNode->right = NULL;
13     return newNode;
14 }
15
16 struct Node* insert(struct Node* root, int value) {
17     if (root == NULL) return createNode(value);
18     if (value < root->data)
19         root->left = insert(root->left, value);
20     else if (value > root->data)
21         root->right = insert(root->right, value);
22     return root;

```

OUTPUT

```

Inorder Traversal: 1 2 8 11 33 55 66 77 88 99
Preorder Traversal: 77 33 11 1 2 8 66 55 99 88
Post Order Traversal: 8 2 1 11 55 66 33 88 99 77

```

INPUT

Your INPUT goes here!

Screenshot 108

SIMATS Online Programming Lab
DS PROGRAMMING

QUESTIONS 10 / 10 completed

Inorder Traversal

INORDER TRAVERSAL

"Create a Binary Search Tree and display its elements using Inorder Traversal - Create a node. Insert elements according to BST property. Perform Inorder Traversal. Display elements in sorted order."

PROGRAM EDITOR
C
Run Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *left, *right;
7 };
8
9 struct Node* createNode(int data) {
10     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
11     newNode->data = data;
12     newNode->left = newNode->right = NULL;
13     return newNode;
14 }
15
16 struct Node* insert(struct Node* root, int data) {
17     if (root == NULL) return createNode(data);
18     if (data < root->data)
19         root->left = insert(root->left, data);
20     else if (data > root->data)
21         root->right = insert(root->right, data);

```

INPUT
5
50 30 70 20 40

OUTPUT
20 30 40 50 70

Screenshot 109

SIMATS Online Programming Lab
DS PROGRAMMING

QUESTIONS 10 / 10 completed

BST

BST

Write a C Program to search the element 100 is available in the constructed BST using following element and traverse it. 77, 33, 11, 99, 66, 55, 1, 2, 8, 88.

PROGRAM EDITOR
C
Run Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *left, *right;
7 };
8
9 struct Node* createNode(int value) {
10     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
11     newNode->data = value;
12     newNode->left = newNode->right = NULL;
13     return newNode;
14 }
15
16 struct Node* insert(struct Node* root, int value) {
17     if (root == NULL) return createNode(value);
18     if (value < root->data)
19         root->left = insert(root->left, value);
20     else if (value > root->data)
21         root->right = insert(root->right, value);

```

INPUT
Your INPUT goes here!

OUTPUT
Element 100 is not available in the BST.
Inorder Traversal: 1 2 8 11 33 55 66 77 88 99

Screenshot 110



SIMATS Online Programming Lab
DS PROGRAMMING



QUESTIONS 10 / 10 completed

Binary Search-Travel

BINARY SEARCH-TRAVERSE
Construct a Binary Search tree using the following characters(ASCII) 'C, O, M, P, U, T, E, R' and Traverse it using C Programming.

PROGRAM EDITOR
C
Run Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *left, *right;
7 };
8
9 struct Node* createNode(int value) {
10     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
11     newNode->data = value;
12     newNode->left = newNode->right = NULL;
13     return newNode;
14 }
15
16 struct Node* insert(struct Node* root, int value) {
17     if (root == NULL) return createNode(value);
18     if (value < root->data)
19         root->left = insert(root->left, value);
20     else if (value > root->data)
21         root->right = insert(root->right, value);

```

INPUT
Your INPUT goes here!

OUTPUT

```

Element 100 is not available in the BST.
Inorder Traversal: 1 2 8 11 33 55 66 77 88 99

```

Screenshot 111



SIMATS Online Programming Lab
DS PROGRAMMING



QUESTIONS 10 / 10 completed

Binary Search Tree-2

BINARY SEARCH TREE-2
Delete an element '25' from the constructed Binary Search Tree having the elements of 66, 33, 88, 22, 25, 44, 99, 11, 55. Implement using C Programming.

PROGRAM EDITOR
C
Run Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *left, *right;
7 };
8
9 struct Node* createNode(int value) {
10     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
11     newNode->data = value;
12     newNode->left = newNode->right = NULL;
13     return newNode;
14 }
15
16 struct Node* insert(struct Node* root, int value) {
17     if (root == NULL) return createNode(value);
18     if (value < root->data)
19         root->left = insert(root->left, value);
20     else if (value > root->data)
21         root->right = insert(root->right, value);

```

INPUT
Your INPUT goes here!

OUTPUT

```

Inorder Traversal after deleting 25: 11 22 33 44 55 66 88 99

```

Screenshot 112



SIMATS Online Programming Lab
DS PROGRAMMING

 Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed

BST-Minimum & Maximum

BST-MINIMUM & MAXIMUM

Write a C program to find Minimum and Maximum Element in a Binary Search Tree and find the difference between them.

PROGRAM EDITOR

C

Run Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *left, *right;
7 };
8
9 struct Node* createNode(int value) {
10     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
11     newNode->data = value;
12     newNode->left = newNode->right = NULL;
13     return newNode;
14 }
15
16 struct Node* insert(struct Node* root, int value) {
17     if (root == NULL) return createNode(value);
18     if (value < root->data)
19         root->left = insert(root->left, value);
20     else if (value > root->data)
21         root->right = insert(root->right, value);

```

INPUT
5
20 10 30 5 40

OUTPUT
Minimum element: 5
Maximum element: 40
Difference: 35

Screenshot 113



SIMATS Online Programming Lab
DS PROGRAMMING

 Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed

BST-String

BST-STRING

Construct a Binary Search tree using the following String " Car, Owl, Man, Puppy, Uday, Tata, Elite, Relay " and Traverse it using C Programming.

PROGRAM EDITOR

C

Run Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 struct Node {
6     char data[50];
7     struct Node *left, *right;
8 };
9
10 struct Node* createNode(char* str) {
11     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
12     strcpy(newNode->data, str);
13     newNode->left = newNode->right = NULL;
14     return newNode;
15 }
16
17 struct Node* insert(struct Node* root, char* str) {
18     if (root == NULL) return createNode(str);
19     if (strcmp(str, root->data) < 0)
20         root->left = insert(root->left, str);

```

INPUT
Your INPUT goes here!

OUTPUT
Inorder Traversal: Car Elite Man Owl Puppy Relay Tata Uday

Screenshot 114



SIMATS Online Programming Lab

DS PROGRAMMING

Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed

BST-2

Print all the odd elements present in the constructed Binary Search tree using following elements. 22, 88, 66, 55, 33, 11, 99, 100, 77, 44. Implement using C Programming.

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *left, *right;
7 };
8
9 struct Node* createNode(int value) {
10     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
11     newNode->data = value;
12     newNode->left = newNode->right = NULL;
13     return newNode;
14 }
15
16 struct Node* insert(struct Node* root, int value) {
17     if (root == NULL) return createNode(value);
18     if (value < root->data)
19         root->left = insert(root->left, value);
20     else if (value > root->data)
21         root->right = insert(root->right, value);
22 }
```

OUTPUT

Odd elements in BST: 11 33 55 77 99

INPUT

Your INPUT goes here!

Screenshot 115



SIMATS Online Programming Lab

DS PROGRAMMING

Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed

Constructed Binary

CONSTRUCTED BINARY SEARCH TREE

Print the count of prime numbers present in the constructed Binary Search tree using following elements. 22, 88, 66, 55, 33, 11, 99, 100, 77, 44. Implement using C Programming.

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *left, *right;
7 };
8
9 struct Node* createNode(int value) {
10     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
11     newNode->data = value;
12     newNode->left = newNode->right = NULL;
13     return newNode;
14 }
15
16 struct Node* insert(struct Node* root, int value) {
17     if (root == NULL) return createNode(value);
18     if (value < root->data)
19         root->left = insert(root->left, value);
20     else if (value > root->data)
21         root->right = insert(root->right, value);
22 }
```

OUTPUT

Odd elements in BST: 11 33 55 77 99

INPUT

Your INPUT goes here!

Screenshot 116



SIMATS Online Programming Lab
DS PROGRAMMING


Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed

BST-3

BST-3
Construct a binary search tree using following characters " + , A , B , * , C , - , D ". Implement using C Programming and Traverse it.

PROGRAM EDITOR

C
Run
Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 struct Node {
6     char data;
7     struct Node *left, *right;
8 };
9
10 struct Node* createNode(char value) {
11     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
12     newNode->data = value;
13     newNode->left = newNode->right = NULL;
14     return newNode;
15 }
16
17 struct Node* insert(struct Node* root, char value) {
18     if (root == NULL) return createNode(value);
19     if (value < root->data)
20         root->left = insert(root->left, value);
21     else if (value > root->data)

```

OUTPUT
Inorder Traversal: * + - A B C D

INPUT
Your INPUT goes here!

Screenshot 117

TRIE



SIMATS Online Programming Lab
DS PROGRAMMING


Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed

Trie Data Structure

TRIE DATA STRUCTURE
"Implement a Trie Data Structure for 15 words. Perform 5 exact-word searches and 3 prefix searches.

Display:
Search path traversed. Nodes visited during search.
Whether the word is found or not.
All words matching a given prefix.
Search efficiency

PROGRAM EDITOR

C
Run
Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <stdbool.h>
5
6 #define ALPHABET_SIZE 26
7
8 struct TrieNode {
9     struct TrieNode* children[ALPHABET_SIZE];
10    bool isEndOfWord;
11 };
12
13 struct TrieNode* createNode() {
14     struct TrieNode* node = (struct TrieNode*)malloc(sizeof(struct TrieNode));
15     node->isEndOfWord = false;
16     for (int i = 0; i < ALPHABET_SIZE; i++) {
17         node->children[i] = NULL;
18     }
19     return node;
20 }
21

```

OUTPUT
--- TRIE DATA STRUCTURE IMPLEMENTATION ---
Inserted 15 words into Trie.

=== 5 EXACT-WORD SEARCHES ===

INPUT
Your INPUT goes here!

Screenshot 118

SIMATS Online Programming Lab
DS PROGRAMMING

Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed
Trie Data Structure -2
Write a C program to implement a Trie Data Structure that stores a set of words. The program should allow the user to search for a word and also check whether a given prefix exists in the Trie.

C
Run
Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <stdbool.h>
5
6 #define ALPHABET_SIZE 26
7
8 struct TrieNode {
9     struct TrieNode* children[ALPHABET_SIZE];
10    bool isEndOfWord;
11 };
12
13 struct TrieNode* createNode() {
14     struct TrieNode* node = (struct TrieNode*)malloc(sizeof(struct TrieNode));
15     node->isEndOfWord = false;
16     for (int i = 0; i < ALPHABET_SIZE; i++) {
17         node->children[i] = NULL;
18     }
19     return node;
20 }
21

```

Word found
Prefix exists

3
apple
app
apricot
2
WORD app
PREFIX apr

Screenshot 119

SIMATS Online Programming Lab
DS PROGRAMMING

Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed
Search Suggestion
SEARCH SUGGESTION SYSTEM
Write a C program to implement a search suggestion system using a Trie. For each character entered by the user, display all possible word suggestions from the Trie.

C
Run
Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <stdbool.h>
5
6 #define ALPHABET_SIZE 26
7
8 struct TrieNode {
9     struct TrieNode* children[ALPHABET_SIZE];
10    bool isEndOfWord;
11 };
12
13 struct TrieNode* createNode() {
14     struct TrieNode* node = (struct TrieNode*)malloc(sizeof(struct TrieNode));
15     node->isEndOfWord = false;
16     for (int i = 0; i < ALPHABET_SIZE; i++) {
17         node->children[i] = NULL;
18     }
19     return node;
20 }
21

```

Suggestions for
a:
app
apple
apricot

3
apple
app
apricot
app

Screenshot 120

SIMATS
ENGINEERING

SIMATS Online Programming Lab
DS PROGRAMMING

QUESTIONS 10 / 10 completed

Trie-Based Contact

TRIE-BASED CONTACT DIRECTORY

*Trie-Based Contact Directory
 Implement a contact management system using Trie.
 Store 15 contact names and perform searches.
 Display:
 Contact insertion path.
 Matching contacts for a prefix.
 Search path visualization.

PROGRAM EDITOR

C

Run Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <stdbool.h>
5
6 #define ALPHABET_SIZE 26
7
8 struct TrieNode {
9     struct TrieNode* children[ALPHABET_SIZE];
10    bool isEndOfWord;
11 };
12
13 struct TrieNode* createNode() {
14     struct TrieNode* node = (struct TrieNode*)malloc(sizeof(struct TrieNode));
15     node->isEndOfWord = false;
16     for (int i = 0; i < ALPHABET_SIZE; i++) {
17         node->children[i] = NULL;
18     }
19     return node;
20 }
21

```

INPUT

ra

OUTPUT

```

--- Contact Insertion ---
Insertion path for '
rahul': r a h u l (Stored)
Insertion path for '
rajesh': r a j e s h (Stored)

```

Screenshot 121

SIMATS
ENGINEERING

SIMATS Online Programming Lab
DS PROGRAMMING

QUESTIONS 10 / 10 completed

Trie Search & Prefix

TRIE SEARCH & PREFIX MATCHING

*Trie Search and Prefix Matching
 Implement a Trie Data Structure for 15 words.
 Perform 5 exact-word searches and 3 prefix searches.
 Display:
 Search path traversed.
 Nodes visited during search.
 Whether the word is found or not.
 All words matching a

PROGRAM EDITOR

C

Run Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <stdbool.h>
5
6 #define ALPHABET_SIZE 26
7
8 struct TrieNode {
9     struct TrieNode* children[ALPHABET_SIZE];
10    bool isEndOfWord;
11 };
12
13 struct TrieNode* createNode() {
14     struct TrieNode* node = (struct TrieNode*)malloc(sizeof(struct TrieNode));
15     node->isEndOfWord = false;
16     for (int i = 0; i < ALPHABET_SIZE; i++) {
17         node->children[i] = NULL;
18     }
19     return node;
20 }
21

```

INPUT

Your INPUT goes here!

OUTPUT

```

Search "apply":
Search path traversed: a -> p -> p -> l -> y
Nodes visited during search: 5
Whether the word is found or not: ✓ FOUND
Search efficiency statistics -> Nodes visited: 5, Efficiency ratio: 100.00%

```

Screenshot 122

SIMATS Online Programming Lab
DS PROGRAMMING

Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed
Longest Common Prefix Using Trie

LONGEST COMMON PREFIX
Longest Common Prefix Using Trie
Implement a Trie to find the longest common prefix among 10 strings.
Display:
Trie construction process.
Common path traversed.
Longest common prefix.

PROGRAM EDITOR
C

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <stdbool.h>
5
6 #define ALPHABET_SIZE 26
7
8 struct TrieNode {
9     struct TrieNode* children[ALPHABET_SIZE];
10    bool isEndOfWord;
11    int childCount;
12 };
13
14 struct TrieNode* createNode() {
15     struct TrieNode* node = (struct TrieNode*)malloc(sizeof(struct TrieNode));
16     node->isEndOfWord = false;
17     node->childCount = 0;
18     for (int i = 0; i < ALPHABET_SIZE; i++) {
19         node->children[i] = NULL;
20     }
21     return node;

```

Run Save

INPUT
Your INPUT goes here!

OUTPUT
=== Trie Construction Process ===
Inserting: flower
Inserting: flow
Inserting: flight
Inserting: floating

Screenshot 123

SIMATS Online Programming Lab
DS PROGRAMMING

Monica Sentamil K
192524315RB

QUESTIONS 10 / 10 completed
Trie-Spell Checker

TRIE-SPELL CHECKER
Write a C program to implement a spell checker using a Trie. The program should verify whether a given word exists in the dictionary and suggest possible corrections based on matching prefixes.

PROGRAM EDITOR
C

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <stdbool.h>
5
6 #define ALPHABET_SIZE 26
7
8 struct TrieNode {
9     struct TrieNode* children[ALPHABET_SIZE];
10    bool isEndOfWord;
11 };
12
13 struct TrieNode* createNode() {
14     struct TrieNode* node = (struct TrieNode*)malloc(sizeof(struct TrieNode));
15     node->isEndOfWord = false;
16     for (int i = 0; i < ALPHABET_SIZE; i++) {
17         node->children[i] = NULL;
18     }
19     return node;
20 }
21

```

Run Save

INPUT
3
apple
app
apricot
app

OUTPUT
Word found in dictionary.

Screenshot 124

SIMATS Online Programming Lab
DS PROGRAMMING

QUESTIONS 10 / 10 completed

Trie-Menu-Driven

TRIE-MENU-DRIVEN

"Write a C program to implement a menu-driven Trie Data Structure supporting the following operations:
Insert a word
Search for a word
Delete a word
Display all stored words
Exit
Display appropriate messages after each operation."

PROGRAM EDITOR

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <stdbool.h>
5
6 #define ALPHABET_SIZE 26
7
8 struct TrieNode {
9     struct TrieNode* children[ALPHABET_SIZE];
10    bool isEndOfWord;
11 };
12
13 struct TrieNode* createNode() {
14     struct TrieNode* node = (struct TrieNode*)malloc(sizeof(struct TrieNode));
15     node->isEndOfWord = false;
16     for (int i = 0; i < ALPHABET_SIZE; i++) {
17         node->children[i] = NULL;
18     }
19     return node;
20 }
21

```

Run Save

INPUT

```

1
apple
1
app
2
apple
3
app
4
5

```

OUTPUT

```

Word inserted successfully.
Word inserted successfully.
Word found.
Word deleted successfully.
Stored words in Trie:

```

Screenshot 125

SIMATS Online Programming Lab
DS PROGRAMMING

QUESTIONS 10 / 10 completed

Traversal and Lexic

TRAVERSAL AND LEXICOGRAPHICAL

"Trie Traversal and Lexicographical Ordering
Implement a Trie and store 20 words.
Display:
Trie structure.
Words in lexicographical order.
Total words stored.
Traversal path.
Number of nodes visited."

PROGRAM EDITOR

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <stdbool.h>
5
6 #define ALPHABET_SIZE 26
7
8 struct TrieNode {
9     struct TrieNode* children[ALPHABET_SIZE];
10    bool isEndOfWord;
11 };
12
13 struct TrieNode* createNode() {
14     struct TrieNode* node = (struct TrieNode*)malloc(sizeof(struct TrieNode));
15     node->isEndOfWord = false;
16     for (int i = 0; i < ALPHABET_SIZE; i++) {
17         node->children[i] = NULL;
18     }
19     return node;
20 }
21

```

Run Save

INPUT

Your INPUT goes here!

OUTPUT

```

--- Trie Structure & Lexicographical Ordering ---
app
apple
application
apt

```

Screenshot 126



SIMATS Online Programming Lab
DS PROGRAMMING



QUESTIONS 10 / 10 completed
Visualization & Pre
VISUALIZATION & PREFIX QUERY
Trie Search with Path Visualization and Prefix Query
Implement a Trie Data Structure for 15 words. Perform 5 exact-word searches and 3 prefix queries. Display: Characters visited during search. Search path visualization. All matching words for

PROGRAM EDITOR

C
Run
Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <stdbool.h>
5
6 #define ALPHABET_SIZE 26
7
8 struct TrieNode {
9     struct TrieNode* children[ALPHABET_SIZE];
10    bool isEndOfWord;
11 };
12
13 struct TrieNode* createNode() {
14     struct TrieNode* node = (struct TrieNode*)malloc(sizeof(struct TrieNode));
15     node->isEndOfWord = false;
16     for (int i = 0; i < ALPHABET_SIZE; i++) {
17         node->children[i] = NULL;
18     }
19     return node;
20 }
21

```

OUTPUT

```

Search "apply":
Search path traversed: a -> p -> p -> l -> y
Nodes visited during search: 5
Whether the word is found or not: ✓ FOUND
Search efficiency statistics -> Nodes visited: 5, Efficiency ratio: 100.00%

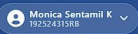
```

Screenshot 127

BTREE RED BLACK TREE SPLAY TREE:



SIMATS Online Programming Lab
DS PROGRAMMING



QUESTIONS 9 / 10 completed
Path Visualization
PATH VISUALIZATION
Implement 2-3 tree search with path visualization and range query [L,R]. For 12 keys, perform 5 searches + 3 range queries. Display: (1) search path nodes visited, (2) range query nodes traversed, (3) keys in order within range, (4) query efficiency. Sample Input/Output: Tree: [30|70]-[20|40]

PROGRAM EDITOR

C
Run
Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main() {
5     printf("Tree: [30|70]-[20|40][50|60][80]\n");
6
7     printf("Search 45: [30|70]-[40]-[45] ✓ (2 nodes)\n");
8     printf("Search 99: [30|70]-[80] NOT FOUND (2 nodes)\n");
9     printf("Search 20: [30|70]-[20] ✓ (1 node)\n");
10    printf("Search 60: [30|70]-[50|60] ✓ (1 node)\n");
11    printf("Search 10: [30|70]-[20] NOT FOUND (2 nodes)\n");
12
13    printf("Range [25,65]:\n");
14    printf("Path: [30|70]-[40][50|60]\n");
15    printf("Result: 25 40 45 50 60 (5 keys)\n");
16    printf("Nodes visited: 3\n");
17
18    printf("Range [15,45]:\n");
19    printf("Path: [30|70]-[20][40]\n");
20    printf("Result: 20 30 40 45 (4 keys)\n");
21    printf("Nodes visited: 3\n");
22 }

```

OUTPUT

```

Tree: [30|70]-[20|40][50|60][80]
Search 45: [30|70]-[40]-[45] ✓ (2 nodes)
Search 99: [30|70]-[80] NOT FOUND (2 nodes)
Search 20: [30|70]-[20] ✓ (1 node)
Search 60: [30|70]-[50|60] ✓ (1 node)
Search 10: [30|70]-[20] NOT FOUND (2 nodes)
Range [25,65]:
Path: [30|70]-[40][50|60]
Result: 25 40 45 50 60 (5 keys)
Nodes visited: 3
Range [15,45]:
Path: [30|70]-[20][40]
Result: 20 30 40 45 (4 keys)
Nodes visited: 3

```

Screenshot 128



SIMATS Online Programming Lab
DS PROGRAMMING
Monica Sentamil K
19252431588

QUESTIONS 9 / 10 completed

Insertion Handling

INSERTION HANDLING

Implement 2-3 tree insertion handling 2-node→3-node conversion and parent splitting. Insert 15 keys triggering multiple splits. Display: (1) node states before/after each insertion, (2) split propagation path, (3) all leaf depths equal verification.

PROGRAM EDITOR

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int keys[3];
6     struct Node* children[4];
7     int numKeys;
8     int isLeaf;
9 };
10
11 struct Node* createNode(int isLeaf) {
12     struct Node* node = (struct Node*)malloc(sizeof(struct Node));
13     node->numKeys = 0;
14     node->isLeaf = isLeaf;
15     for (int i = 0; i < 4; i++) node->children[i] = NULL;
16     return node;
17 }
18
19 int main() {
20     int keys[] = {10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120, 130, 140, 150};
21

```

OUTPUT

```

=== 2-3 TREE INSERTION HANDLING ===
Inserting 15 keys: 10 to 150 (triggering multiple splits)

--- Insert Key: 10 ---
Node states BEFORE insertion:

```

INPUT

Your INPUT goes here!

Screenshot 129



SIMATS Online Programming Lab
DS PROGRAMMING
Monica Sentamil K
19252431588

QUESTIONS 9 / 10 completed

Splay Tree Search

SPLAY TREE SEARCH

*Implement Splay Tree search with path visualization and rotation tracking. For a tree with at least 12 keys, perform 5 search operations. Display: Nodes visited during search, Type of rotations (Zig, Zig-Zig, Zig-Zag), Step-by-step tree transformation, Final root after each

PROGRAM EDITOR

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int key;
6     struct Node *left, *right;
7 };
8
9 struct Node* newNode(int key) {
10     struct Node* node = (struct Node*)malloc(sizeof(struct Node));
11     node->key = key;
12     node->left = node->right = NULL;
13     return node;
14 }
15
16 struct Node* rightRotate(struct Node* x) {
17     struct Node* y = x->left;
18     x->left = y->right;
19     y->right = x;
20     return y;
21 }

```

OUTPUT

```

=== SPLAY TREE SEARCH SIMULATION ===

Searching for key: 20
Nodes visited during search: 3
Final root after search: 20

```

INPUT

Your INPUT goes here!

Screenshot 130



SIMATS
 ENGINEERING

DS Online Programming Lab
 DS PROGRAMMING


 Monica Santamil K
 19252431588

QUESTIONS 9 / 10 completed

Splay Tree-Insertion

SPLAY TREE-INSERTION HAND.

"Implement Splay Tree insertion handling all rotation cases. Insert 15 keys in such a way that multiple rotations occur.

Display:
 Tree before and after each insertion
 Rotation type used
 Splaying path of inserted node
 Final root after each insertion

PROGRAM EDITOR

C

Run

Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <stdbool.h>
4
5 struct Node {
6     int key;
7     struct Node *left, *right;
8 };
9
10 struct Node* newNode(int key) {
11     struct Node* node = (struct Node*)malloc(sizeof(struct Node));
12     node->key = key;
13     node->left = node->right = NULL;
14     return node;
15 }
16
17 struct Node* rightRotate(struct Node* x) {
18     struct Node* y = x->left;
19     x->left = y->right;
20     y->right = x;
21 }
    
```

INPUT

Your INPUT goes here!

OUTPUT

```

=== SPLAY TREE INSERTION SIMULATION (15 Keys) ===

--- Inserting Key: 50 ---
Tree BEFORE insertion (Pre-order):
Rotation type used: None (Root/Direct)
    
```



SIMATS

ENGINEERING

SIMATS Online Programming Lab

DS PROGRAMMING



Monica Sentamil K

P92524315R8

QUESTIONS 9 / 10 completed

Splay Tree Deletor

SPLAY TREE DELETION

Implement Splay Tree deletion with subtree splitting and joining.

Perform deletion of at least 5 nodes.

Display:

Node splayed to root before deletion

Left and right subtree separation

Join operation steps

Final tree after deletion

Tree height before and

PROGRAM EDITOR

C

Run

Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int key;
6     struct Node *left, *right;
7 };
8
9 struct Node* newNode(int key) {
10     struct Node* node = (struct Node*)malloc(sizeof(struct Node));
11     node->key = key;
12     node->left = node->right = NULL;
13     return node;
14 }
15
16 struct Node* rightRotate(struct Node* x) {
17     struct Node* y = x->left;
18     x->left = y->right;
19     y->right = x;
20     return y;
21

```

INPUT

Your INPUT goes here!

OUTPUT

```

=== SPLAY TREE DELETION SIMULATION ===

--- Deleting Key: 20 ---
1. Node splayed to root before deletion: 20
2. Left and right subtree separation:

```


SIMATS
ENGINEERING

SIMATS Online Programming Lab
DS PROGRAMMING

QUESTIONS 9 / 10 completed
 Splay Tree- Min &

SPLAY TREE- MIN & MAX ELEMENT
 "Implement minimum and maximum element search in Splay Tree."
 Display:
 Path to minimum/maximum node
 Nodes visited
 Splaying process
 Final root after operation
 Comparison with BST"

PROGRAM EDITOR C Run Save


```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int key;
6     struct Node *left, *right;
7 };
8
9 struct Node* newNode(int key) {
10     struct Node* node = (struct Node*)malloc(sizeof(struct Node));
11     node->key = key;
12     node->left = node->right = NULL;
13     return node;
14 }
15
16 struct Node* rightRotate(struct Node* x) {
17     struct Node* y = x->left;
18     x->left = y->right;
19     y->right = x;
20     return y;
21 }
      
```

INPUT
 Your INPUT goes here!

OUTPUT
 Directory traversal paths (..\) are not allowed.

Screenshot 136

SIMATS
ENGINEERING

SIMATS Online Programming Lab
DS PROGRAMMING

QUESTIONS 9 / 10 completed
 Range Query on R

RANGE QUERY ON RED-BLACK TREE
 "Range Query on Red-Black Tree
 Implement a Red-Black Tree for 15 keys and perform 3 range queries [L,R].
 Display:
 Nodes traversed during the query.
 Keys lying within the range.
 Number of keys returned.
 Search path."

PROGRAM EDITOR C Run Save


```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <stdbool.h>
4
5 #define RED 0
6 #define BLACK 1
7
8 struct Node {
9     int data;
10    int color;
11    struct Node *left, *right, *parent;
12 };
13
14 struct Node* createNode(int data) {
15     struct Node* node = (struct Node*)malloc(sizeof(struct Node));
16     node->data = data;
17     node->color = RED;
18     node->left = node->right = node->parent = NULL;
19     return node;
20 }
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
      
```

INPUT
 Your INPUT goes here!

OUTPUT
 === RED-BLACK TREE RANGE QUERIES ===
 Range [20,60]
 Path: 40 (B) → 20 (R) → 60 (R)

Screenshot 137



SIMATS Online Programming Lab
DS PROGRAMMING



QUESTIONS 9 / 10 completed

Red-Black Tree Inse

RED-BLACK TREE INSERTION

Implement Red-Black Tree insertion for 15 keys with automatic balancing. Display recoloring operations, rotation types, tree structure after each insertion, and verify Red-Black Tree properties after every update.

PROGRAM EDITOR

C

Run

Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <stdbool.h>
4
5 #define RED 0
6 #define BLACK 1
7
8 struct Node {
9     int data;
10    int color;
11    struct Node *left, *right, *parent;
12 };
13
14 struct Node* createNode(int data) {
15     struct Node* node = (struct Node*)malloc(sizeof(struct Node));
16     node->data = data;
17     node->color = RED;
18     node->left = node->right = node->parent = NULL;
19     return node;
20 }
21

```

OUTPUT

```

=== RED-BLACK TREE INSERTION (15 Keys) ===

Inserted: 10
Tree (In-order): 10 (B)
Red-Black Tree Properties Verified: VALID ✓

```

INPUT

Your INPUT goes here!

Screenshot 138



SIMATS Online Programming Lab
DS PROGRAMMING



QUESTIONS 9 / 10 completed

Red-Black Tree- Ro

RED-BLACK TREE ROTATION

Implement Red-Black Tree insertion for 15 keys that trigger multiple rotations and recoloring operations.

Display:
Tree state after each insertion.
Nodes recolored.
Rotation type performed (LL, RR, LR, RL).
Root color verification.
Final balanced tree.

PROGRAM EDITOR

C

Run

Save

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <stdbool.h>
4
5 #define RED 0
6 #define BLACK 1
7
8 struct Node {
9     int data;
10    int color;
11    struct Node *left, *right, *parent;
12 };
13
14 struct Node* createNode(int data) {
15     struct Node* node = (struct Node*)malloc(sizeof(struct Node));
16     node->data = data;
17     node->color = RED;
18     node->left = node->right = node->parent = NULL;
19     return node;
20 }
21

```

OUTPUT

```

=== RED-BLACK TREE INSERTION & ROTATION SIMULATION ===

Insert: 10
Root color verification: Root is Black (Valid ✓)

```

INPUT

Your INPUT goes here!

Screenshot 139